SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Training cycle for engineers in Telecommunications

Option :

SNI

Graduation Project Report

Theme :

« Design and development of a generic configuration management model and continuous deployment of multiple applications/micro services on a Kubernetes cluster»

Realized by :

Ameni ELLOUZE

Work proposed and carried out in collaboration with:

Sofrecom
The Know-How Network

Supervised by:

Mr.*Meher Heni* — ***Supcom Supervisor***
Mr.*Wael Abidi* — ***Sofrecom Supervisor***

Academic Year: 2020-2021

Abstract

The present report is written as a part of the graduation project at the Higher School of Communication of Tunis (SUPCOM). This project was carried out at Sofrecom Tunisia.
The objective of this project is to develop a production-ready kubernetes cluster: and to implement of a Continuous Delivery Pipeline with ArgoCD for the automatic deployment of applications and finally the implementation of a monitoring platform with Prometheus and Grafana to ensure high availability of the kubernetes cluster.

Keywords: GitOps, Docker, Kubernetes, ArgoCD, Helm Chart, Automation, Prometheus, Grafana, ...

Résumé

Le présent rapport est rédigé dans le cadre du projet de fin d'études de l'Ecole Supérieure de Communication de Tunis (SUPCOM). Ce projet a été réalisé à Sofrecom Tunisie.
L'objectif de ce projet est de développer un cluster kubernetes prêt pour la production et la mise en place d'un pipeline de Livraison Continue avec ArgoCD pour le déploiement automatique des applications et finalement la mise en place d'une plateforme de monitoring avec Prometheus et Grafana pour assurer haute disponibilité du cluster kubernetes.

Mots-clés: GitOps, Docker, Kubernetes, ArgoCD, Helm Chart, Automation, Prometheus, Grafana, ...

Dedications

“

To my grandparents, Mohamed and Nadhima, who had always supported me, The words are never enough to thank you for your eternal support, your prayers, for everything. May God Bless You with Good Health and Long Life,

To my parents, Chokri and Sonia, so many words cannot express my love and affection. You have always supported and encouraged me throughout my journey. On this memorable day, for me as well as for you, receive this work as a sign of my deep appreciation and my profound esteem,

To my brother and sister in law, Ahmed and Fatma, I can never imagine how the world would be without you. Both of you have been my best cheerleaders. Wishing you all luck and success in your lives,

To my nephew Mohamed and my niece Maram, Thank you for bringing the joy and light even to my darkest days,

To someone close to my heart, someone who was there for me in my ups and downs, in my joy and sadness. To Adib, I cannot express my deepest gratitude for the continuous support you have always shown. Thank you for everything you have added to my life.

To all my dear ones, to all of you

Thank you.

”

- Ameni

Acknowledgement

I would like to express my gratitude to all who contributed and assisted me in completing this mission. My 18 years of studies had come to a successful conclusion thanks to you.

To **Mr. Maher Heni**, my academic supervisor, thank you for your support and oversight, as well as your wise words and suggestions. Thank you for being my key source of help through this internship and during my engineering scholarship. Thank you so much.

To **Mr. Wael Abidi** my industrial supervisor. Thank you for all the lessons you've given me, for teaching me the best way to do it, and for handing me the keys to the technical world, Without your advice and encouragement, I would not have made it this far.

Thanks to all of the SUPCOM teachers who were involved in my education during my time at university. And thank you to the Orange Labs Tunisia Sofrecom team for welcoming me as a member of the team and providing me with the opportunity to learn from them. I would like to express my gratitude to the members of the jury for the honor they have bestowed upon me by reviewing my graduation project, and I hope that they can find the clarification and inspiration that they seek in this report.

Contents

Abstract . . . . . **I**

Résumé . . . . . **II**

Dedications . . . . . **III**

Acknowledgement . . . . . **IV**

General Introduction **1**

1 General Presentation of the project **3**
1.1 The host enterprise 4
1.1.1 Enterprise presentation 4
1.1.2 Sector of activity 4
1.1.3 Sofrecom Tunisia 5
1.2 Context of the project 5
1.2.1 Study of the current solution 5
1.2.2 Objectives and contributions 6
1.3 Development method 6
1.3.1 The Agile method 6
1.3.2 The chosen methodology: Kanban 7
1.4 Conclusion 8

2 State of the art **9**
2.1 Git 10
2.2 DevOps 11
2.2.1 Principles 11
2.2.2 DevOps benefits 13
2.2.3 DevOps tools 14
2.3 GitOps 15
2.3.1 Principles 15
2.3.2 Benefits 16
2.3.3 Tools 17
2.3.4 Push-based pipeline 17
2.3.5 Pull-based pipeline 18
2.4 Virtualisation 18
2.5 Microservice architecture 19
2.6 Monitoring 20

2.7 Conclusion . . . 21

3 Analysis and Conception . . . 22
3.1 Specification of needs . . . 23
3.1.1 Actors . . . 23
3.1.2 Functional needs . . . 23
3.1.3 Non-functional needs . . . 24
3.1.4 Proposed architecture . . . 24
3.2 Working environment . . . 25
3.2.1 Hardware environment . . . 25
3.2.2 Software environment . . . 26
3.3 Conclusion . . . 31

4 Kubernetes Infrastructure Automation . . . 32
4.1 Use case diagram . . . 33
4.2 Sequence diagram . . . 34
4.3 Realization . . . 35
4.3.1 Deploying the infrastructure manually . . . 35
4.3.2 Automating the deployment . . . 35
4.4 Conclusion . . . 38

5 Orchestrations . . . 39
5.1 Kubernetes Resources for Deployment . . . 40
5.2 Templates for Deployment Files: HELM . . . 40
5.3 ArgoCD installations and configurations . . . 42
5.3.1 Installation . . . 42
5.3.2 Configuration . . . 42
5.3.3 Where GitLab meets ArgoCD . . . 48
5.4 Conclusion . . . 49

6 Monitoring . . . 50
6.1 Deployment Diagram . . . 51
6.2 Realization . . . 52
6.2.1 Collecting data with Prometheus . . . 52
6.2.2 Sending notifications to the AlertManager . . . 55
6.2.3 Visualizing data with Grafana . . . 57
6.3 Conclusion . . . 59

General Conclusion . . . 60

Netography . . . 64

Appendices . . . 65

A Kubernetes . . . 66

B ArgoCD . . . 70

List of Figures

1.1 Sofrecom around the world 4
1.2 Kanban events [3] 8

2.1 Continuous Integration[7] 12
2.2 Continuous Deployment and Continuous Delivery[8] 13
2.3 DevOps Process[10] 14
2.4 Push-based pipeline [13] 17
2.5 Pull-based pipeline[14] 18
2.6 VMs vs Container [16] 19
2.7 Microservices vs Monolithic architecture [18] 20

3.1 Global use case diagram 24
3.2 Global architecture 25
3.3 Dockers logo[20] 26
3.4 Gitlabs' logo[22] 26
3.5 Kubernetes' logo[23] 27
3.6 GitlabCIs' logo[24] 27
3.7 ArgoCD's logo[26] 27
3.8 Lens' logo[28] 27
3.9 Grafana logo [30] 27
3.10 Prometheus logo [32] 28
3.11 AlertManager logo[33] 28
3.12 Helm logo [35] 28
3.13 Nginx's logo[37] 28
3.14 Ansible logo [39] 29
3.15 MariaDB logo [41] 29
3.16 OpenEBS logo [40] 29
3.17 Visual Studio Code logo [44] 29
3.18 Bash Scripting logo [46] 30
3.19 Draw io logo [47] 30
3.20 Microsoft PowerPoint logo[48] 30

4.1 Automate infrastructure deployment use case 33
4.2 Automate infrastructure deployment sequences diagram 34
4.3 Infrastructure initial architecture 35
4.4 Kubernetes Ansible project content 36
4.5 Kubernetes Ansible project output of the adding of the worker nodes . . . 36
4.6 Infrastructure automated architecture 37
4.7 Dashboard added namespace 37

4.8 Nginx deployment on Kubernetes . . . 38
4.9 Kubernetes dashboard . . . 38

5.1 Example of Repetitive YAML resources . . . 41
5.2 Faster and Smarter Deployments with HELM and ArgoCD . . . 41
5.3 Frontend Helm Chart . . . 41
5.4 ArgoCD's pods running . . . 42
5.5 ArgoCD's Dashboard . . . 43
5.6 Frontend Helm Gitlab repository . . . 43
5.7 Connect repo using HTTPS . . . 44
5.8 Helm Chart section . . . 44
5.9 The application deployed . . . 45
5.10 Kubernetes objects within ArgoCD's application . . . 45
5.11 Yaml configuration . . . 46
5.12 Orange Gitlab container registry . . . 46
5.13 The frontend of the application . . . 47
5.14 Let's Encrypt YAMLS file . . . 47
5.15 Cert-manager Pods . . . 48
5.16 GitLab CI meets ArgoCD . . . 48
5.17 CI Pipeline . . . 49

6.1 Monitoring deployment diagram . . . 51
6.2 The architecture of Prometheus . . . 52
6.3 Prometheus dashboard . . . 53
6.4 Prometheus Targets displayed in a Table . . . 54
6.5 Prometheus Targets displayed in a Graph . . . 54
6.6 Prometheus Targets . . . 55
6.7 Prometheus deployed . . . 55
6.8 AlertManager dashboard . . . 56
6.9 AlertManager dashboard . . . 56
6.10 Slack alerting message . . . 57
6.11 AlertManager Silence . . . 57
6.12 Grafana application deployed . . . 58
6.13 Grafana logging dashboard . . . 58
6.14 Grafana data source . . . 59
6.15 Grafana dashboard . . . 59

A.1 Kubernetes adoption . . . 66
A.2 Kubernetes architecture . . . 67

B.1 ArgoCD keeps the live state synchronised with the desired state . . . 71
B.2 ArgoCD web UI example . . . 73
B.3 ArgoCD Architecture . . . 73
B.4 ArgoCD Pods . . . 73

List of Tables

4.2 Description of automate infrastructure deployment use case 33

List of acronyms

API	*Application Programming Interface*
CAAS	*Container as A Service*
CD	*Continuous Deployment*
CI	*Continuous Integration*
CPU	*Central Processing Unit*
IAAS	*Infrastructure as A Service*
K8S	*Kubernetes*
OLT	*Orange Labs Tunisial*
PAAS	*Platform as A Service*
PVC	*Persistent Volume Claim*
PV	*Persistent Volume*
YAML	*Yet Another Markup Language*
RAM	*Random Access Memory*
UML	*Unified Modeling language*
VM	*Virtual Machine*
MTTR	*Mean Time To Recovery*
CRD	*Custom Resource Definition*

General Introduction

The IT world is continually changing. Infrastructures in the mid-1990s comprised solely of hardware acquired by the corporation and maintained within its own premises. Some specialized operators had to physically maintain the physical servers.
The servers were handled, in this context, utilizing the mutable infrastructure technique, which entails keeping and modifying the server in place. When it came to software development, code was developed in isolation at the time.
There was no organized versioning. The development team handled code compilation and testing by hand, as well as the creation of release packages. The release package was then handed over to the operations team, who would be in charge of delivering it. There was a varied time lag, ranging from a few hours to several days, before the new release got into production. This time was determined by the operations team's availability and expertise. Furthermore, there was a lack of communication and collaboration between the development and operations teams. The development team was dissatisfied due to the possibility of extended deployment time frames for new versions. The operations team, on the other hand, was irritated if the new release caused issues when it was deployed.

Employee unhappiness occurred, as did slow software development and release schedules. The Agile technique took popularity in the early 2000s. It enhanced the development process but not the deployment process, indicating that something was still lacking. As a result of a discussion about the shortcomings of the Agile strategy, the notion of DevOps originated around 2008. DevOps is a mindset that has enabled the automation and integration of processes between software development and operations teams via the use of techniques and technologies.

From that point forward, the software may be produced, tested, and published in a more timely and dependable manner. It narrowed the gap between the development and operations teams. CI/CD pipelines and Infrastructure as Code are two of the many new features provided. Returning to the topic of changeable infrastructures, there have been significant advancements in this field as well. The introduction of virtualization was the first enhancement. It is a technique that enabled the development of virtual machines first, then containers (lightweight virtual machines) afterward.

Companies may now construct virtual servers instead of actual ones thanks to VMs. It enables a single physical computer to host numerous server instances. The immutable infrastructure concept arose in this context. It indicates that once a server has been provided, it will never longer be changed. If a new version is required, a new virtual server will be created and the old one will be decommissioned.

The GitOps method was inspired by cloud computing, DevOps best practices, containers, and container orchestrators such as Kubernetes. GitOps, in a nutshell, is concerned with Continuous Deployment in a cloud-native context. It was designed from the start to operate with Kubernetes. It introduces Git as a single source of truth, and Git repositories are used to hold declaratively defined infrastructure files, as part of the Infrastructure as Code methodology. One of the most significant benefits of GitOps is the pull-based pipeline, which is made feasible by operators such as ArgoCD or Flux.

Cloud native apps are becoming increasingly widespread, and Kubernetes is often used to manage them. Because daily deployments might be hundreds of times, developers must be more autonomous in infrastructure administration in addition to focusing on the source code. Automation of the procedures of provisioning and upgrading of k8s environments linked to features, releases, and production will boost productivity and cooperation between development and operations teams, reducing deployment times and, as a result, the development life cycle.

As a consequence, the operations team is freed of a significant load and can devote its time to other activities that provide value to the firm. Developers may work with the infrastructure without needing to understand how Kubernetes works or what commands are required. They can create and configure Kubernetes setups without having to write infrastructure YAML files.

This report contains a comprehensive summary of all of the tasks completed as part of my graduation project for an engineer's degree from Tunis's Higher School of Communication (SUP'COM). This study is divided into six chapters: We begin by introducing the general context of the project, namely the host organisation and the scope of the project. Then we study the State of the Art of the solution as well as some technical concepts for CI/CD pipelines. After that, we detail how we set up our working environment. Then we detailed the steps of our project: The kubernetes infrastructure automation, the orchestrations, and the monitoring. Finally, we end up this report with a conclusion and prospects.

Chapter 1

General Presentation of the project

Introduction

In this chapter, we will describe the host organization where we worked on our graduation project. In order to reveal the goals of our work, We will also describe the project's context, current solution, and problems. Finally, We'll go over the architecture process that was used.

1.1 The host enterprise

In this section, we will present the host enterprise and its activity sector.

1.1.1 Enterprise presentation

Sofrecom is a subsidiary of the Orange Group: it has developed the know-how for 50 years in the operator's field, making it a world leader in consulting and telecommunications engineering. The group accompanies its customers (operators, industrials, governments…) in their project from its management, creation, recovery, or transformation based on its sector of activity. Sofrecom group has today more than 2000 consultants and experts distributed on 11 sites worldwide, operating on 4 continents through its subsidiaries and branches to satisfy more than 200 customers in more than 100 countries.

The Figure 1-1 describes the geographic spread of Sofrecom around the world.

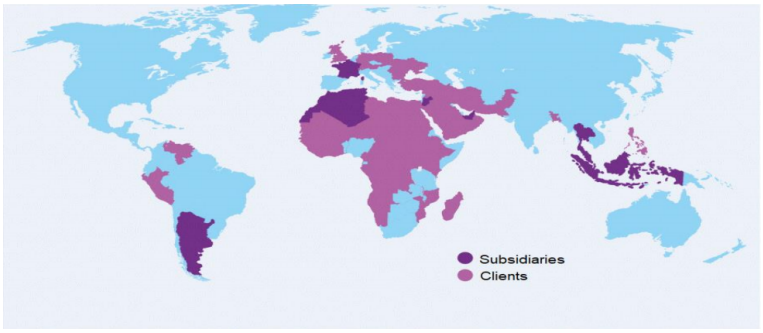


Figure 1.1: Sofrecom around the world

1.1.2 Sector of activity

Sofrecom was able to confirm its experience in several sectors like:

- Consulting
- Innovation
- Development
- Engineering

- Security
- Architecture
- Support and Maintenance

1.1.3 Sofrecom Tunisia

Sofrecom Tunisia is a major actor in consulting and engineering in telecommunications on the local market. Initiated in October 2011, Sofrecom Tunisia, the youngest subsidiary of the group, has today more than 950 engineers and experts. Sofrecom Tunisia has acquired its strength and credibility through hundreds of internationally diverse projects allowing it to be one step ahead of its competitors.

Host department

This internship took place in Orange Labs Tunisia (OLT) department of Sofrecom, which was created in November 2011 to be a link between Europe and Africa. OLT has today more than 160 experts, architects, project managers, system and integration engineers. Its mission is to contribute to the implementation of the OLPS strategic plan and the realization of its product "roadmaps". And it contributes to major projects for the Orange group, among which we mention: Cloud PRO, OIDC, Zebra, Orange Money, PROMISE, USSO, Orange Money, OOPRO.

1.2 Context of the project

In this section, we will study our project in the contest of the actual solution. Then we will pass to figure out the current problematics and the motivation to implement a new and optimized solution.

1.2.1 Study of the current solution

In this part, we first present the procedure that OLT Sofrecom followed initially for the deployment of its applications, then we expose the problematic. To manage the life cycle of its applications before their deployment, Sofrecom had a continuous integration chain based on "Gitlab CI".
Although OLT already has a continuous integration chain, it remains insufficient for their needs. Each application is deployed then using container deployment by using a management software like docker, but managing containers without an orchestrator is difficult. If a developer needs a Kubernetes environment to work on a new feature, he must request it from a member of the operations team. Several days or even weeks may elapse between the time the request is made and the environment is supplied. The operations staff then schedules the commitment, and when the time arrives, more time is required to complete the provisioning. If it is not an automated procedure, it will take time to complete.

1.2.2 Objectives and contributions

The objective of our project is to set up a production-ready kubernetes cluster, as well as to develop a monitoring dashboard displaying the performance metrics of the previously set up pipeline.
Moving to a cloud computing environment is a step to ensure more availability, scalability and likely cost efficiency.
We take the deployment of a pilot application named "WASSUP": Application to integrate services with the Wassup enabler (Orange France internal enabler), with a Frontend developed with Angular and a Backend with SpringBoot and a MariaDB database, to verify the proper functioning of our solution. Although the continuous integration pipeline already exists on GitLab, a change to the pipeline is necessary.
In addition, with continuous deployment we want to automatically deploy any changes made that pass a battery of tests previously set up. In order to set up this pipeline to meet the last two points, we used Ansible scripts, which is a remote computer management tool, to deploy a Kubernetes origin cluster that will be our continuous deployment platform. The development of a "pull-based" GitOps pipeline, using ArgoCD as a continuous deployment operator and GitLab Actions for continuous integration and other automated processes, is part of the project effort.
With this pipeline, Sofrecom customers no longer have to download artifacts and install applications, they simply have access to a URL, reducing a customer's wait time from hours to minutes. Once these two pipelines (continuous integration and continuous deployment) are set up and integrated, a dashboard allowing monitoring (Prometheus-Grafana) and control of the systems will be displayed as needed.

1.3 Development method

In this section, we will specify the development methodology that we have implemented during the realisation of this project.

1.3.1 The Agile method

During this project, we have used Agile as a development method. Agile is a type of project management process that promotes continuous iteration of development and testing through the software development lifecycle of the project. It helps team members to react quickly to all the feedback that they receive about their project. It gives the chance to apply continuous changes and improvements and redirects the project during the development cycle through regular meetings called sprints. Agile software development is based on four main values:

1. Focusing on individual and team interactions rather than processes and tools.

2. Focusing on delivering working software rather than comprehensive documentation.

3. Concentrating on customer collaboration over contract negotiation.

4. Permitting responding to change rather than following a fixed plan.

Agile development offers many benefits. They include:

- **Stakeholder engagement and satisfaction:** Agile development encourages the stakeholder's engagement through regular meetings.
  It helps the developers to better understand the project's requirements and the clients' exact needs.
- **Transparency:** The agile method focuses on engaging the client through each step of the development process. This engagement gives the client a clear and complete vision of the product from early stages.
- **The ability to change:** To give the best expected results, Agile gives the chance to change the plan at any point of the development.
- **Giving purpose to team members:** Since that teams with purpose are more productive, members will challenge themselves to be more and be more efficient. Thus, the agile process focuses on giving a shared sense of ownership and goals for the team members.

1.3.2 The chosen methodology: Kanban

To achieve the main goals of the project, the Kanban [1] development methodology of Agile seems to be the most suitable in order to ensure the smooth running of the project.

Kanban Principles

- **Change Management Principles:**
  Kanban is structured to address the human tendency to resist change.
  - Begin with what we're doing right now: Recognize and respect existing roles, responsibilities, and job titles, as well as understand present procedures as they are actually performed.
  - Accept the goal of continuous progress through evolutionary change.
  - Encourage leadership at all levels.
- **Service Delivery Principles:**
  These principles recognize that organizations are a collection of interrelated services and that the focus should be on the work rather than the individuals doing the job.
  - Recognize and prioritize our consumers' needs and expectations.
  - Manage the task while allowing others to self-organize around it.
  - Improve consumer and business outcomes through evolving policies.

Kanban events

The Kanban approach specifies seven cadences[2], or rituals. Cadences are the cyclical assessments that drive evolutionary change and successful service delivery. The figure below depicts the seven cadences as well as the proposed frequency for modifications in common systems.

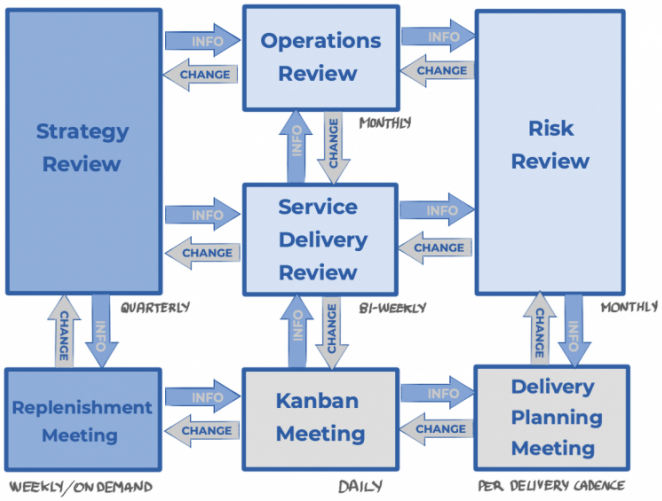


Figure 1.2: Kanban events [3]

Strategy review: establish the notion of "fit for purpose" in context and detect the external environment in order to offer direction to services.
Operations review: understand the balance between services, implement resources to maximize the delivery of value to customer expectations.
Risk review: to understand the risks for the effective provision of services.
Service Delivery Review: examine and improve the effectiveness of a service.
Replenishment meeting: to move elements through the commitment point and supervise the preparation of options for selection in the future.
The Kanban meeting: the daily review of coordination, self-organization, and planning for those who collaborate to provide the service. It often uses a "stand-up" format to encourage a short energetic meeting with the focus on finalizing work items and unlocking issues.
Delivery planning meeting: monitoring and delivery planning to customers

1.4 Conclusion

Through this chapter, we presented the host company Sofrecom and specified its sector of activity. We introduced the project, the current solution and identified the objectives and contributions and identified the adopted methodology to implement it and reach the project's goals. In order to achieve these goals, we will explain, in the next chapter, the GitOps methodology as well as the main concepts that will help us to realize our project through a state of the art study.

Chapter 2

State of the art

Introduction

In this part, we will go through the most critical factors that play a role in our solution's execution. We will begin with the GitOps approach, which serves as the foundation for our solution. Then we will go on to virtualisation and then to the microservices architecture that we used to build the approach. Finally, we will emphasize how critical monitoring is to our approach.

Towards GitOps

2.1 Git

Git [4]is an open source distributed version control system. It is principally used for managing source code in software projects, but it can also be used for generic files.

Version Control

Version control is a mechanism for tracking changes to one or more files. Versions are always available, and if a user wishes to revert to a prior state of files managed by Git, he can do so.

Distributed Version Control System

Firstly there was the Local Version Control System, which used a basic database to keep all changes to a file under revision control. The Centralized Version Control System, which added the collaboration capability, was the next step in the progression.
This function is made possible by a single centralized server that houses all versioned data. However, this system has the issue of a single point of failure. Nobody can work on it if the centralized server goes down for an hour. Then there's the Distributed Version Control System, which is Git's method.

Git handles its data as a series of snapshots, taking a picture of what all files look like at every commit and storing a reference to that snapshot. If the files have not changed, Git does not save them anew, instead of using a link to the previous ones it has already stored for efficiency reasons.
To share a user's work with other team members, a connection to the repository located on a network remote server is required. The remote repository has the entire history of all files.

File states

A file can be in one of three states: changed, staged, or committed.

- Changed signifies that files have been updated but have not yet been committed to the local database.

- Staged indicates that a modified file in its current version has been tagged to go into the next commit snapshot.
- Committed implies that the data has been safely placed in the local database.

Commit

A commit is a list of all modifications made since the parent commit; it functions similarly to a snapshot.
Furthermore, commits include certain metadata information connected with them, such as a unique id that identifies the commit within the repository. Every commit must be accompanied by a message that describes the changes.

Branch

A branch is just a series of commits that follow one another. It is nothing more than a reference to a certain commit.
Any commit can be ”branched” into a new branch, resulting in a new branch from that point forward.
It is a series of one or more consecutive commits. In general, there is always a starting branch known as master.

2.2 DevOps

The term ”DevOps” [5] refers to a set of philosophies that have come together to form a modern production theory. Devops is indeed an innovative modern movement that arose from the intersection of two main trends:

1. The first, also known as ”agile architecture” or ”agile processes,” arose from the application of Agile and Lean methodologies to operations work.
2. The second is a better appreciation of the importance of teamwork between the Development (Dev) and Operations (Ops) teams in order to design, test, and deploy applications more quickly and efficiently. This pace allows businesses to help meet the demands of their clients and perform more effectively in the industry.

The developments and operations departments are no longer isolated in the DevOps strategy. Both the Dev and Ops departments have been combined into a single unit that works together on the whole product lifecycle, from development to testing and implementation. Instead of relying on a single talent, this team must cultivate a diverse set of abilities.

2.2.1 Principles

DevOps is associated with a series of principles [6]

Automation

DevOps emphasizes the need for automating as much as possible. Automation includes tasks like development, testing, configuration, and deployment. If properly done, it eliminates the possibility of human mistakes, increasing dependability. There is also an improvement in agility and repeatability. Developers and operations teams may focus on other tasks, such as increasing the product's value and quality.

Continuous Integration

Since developers often send code changes, it is essential to automate the process of creating and testing. This is accomplished through the use of pipelines and Continuous Integration. It enables the detection of issues early on and the maintenance of code in a deployable form.

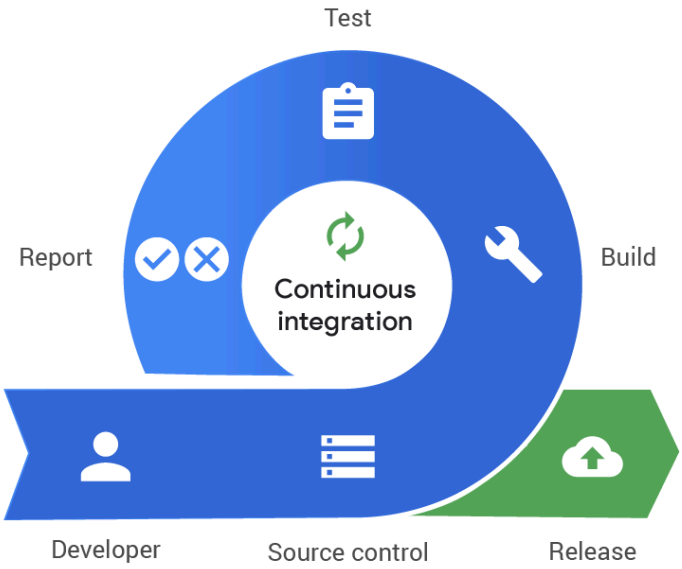


Figure 2.1: Continuous Integration[7]

Continuous Deployment/Delivery

It is a result of the CI. When the pipeline builds and verifies the code changes, Continuous Deployment/Delivery automatically pushes those changes to production.
It enables a speedier time to market, a more consistent deployment procedure, and dependable rollbacks.
Continuous Delivery and Continuous Deployment are ideas that are sometimes confused. Both share the abbreviation CD and can be used in conjunction with CI. The most significant distinction is in the deployment procedure.
Code flows automatically through various processes in Continuous Delivery to prepare it for production deployment, but it does not go live automatically. The code modifications

must be manually authorized first.
Instead, with Continuous Deployment, the code is automatically pushed into a live production environment, with the newly deployed environment being monitored to see if there are any issues. In that instance, a rollback is performed. Figure 2-2 depicts the distinction between continuous deployment and continuous delivery:

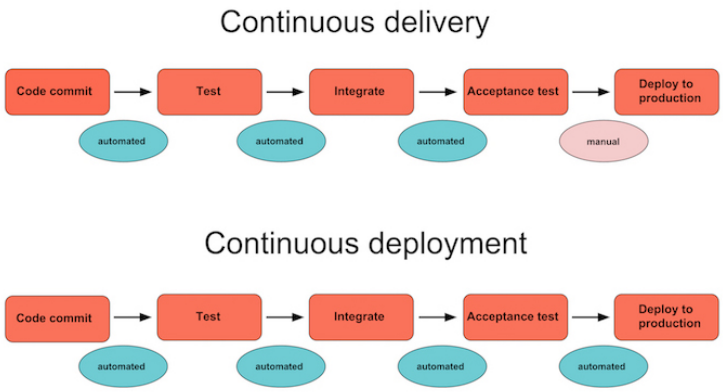


Figure 2.2: Continuous Deployment and Continuous Delivery[8]

Infrastructure as Code

IaC [9] enables the management and provisioning of IT infrastructure using code and automation. Consistent resource creation and management, reusability, scalability, self-documented infrastructure, and simplification of complicated infrastructures are all advantages of the IaC. "Infrastructure as Code" is defined as "the management of infrastructure in a descriptive model, using the same versioning as the DevOps team employs for source code." The IaC paradigm creates the same environment every time it is used, similar to how source code creates the same binary every time it is built. Without IaC, each target environment must be managed separately, which becomes a difficult and complex operation, especially when there are several targets.

Continuous Monitoring

This method guarantees that the application operates smoothly while also gathering data on the performance and stability of services, systems, and infrastructures. It enables the detection of issues and rapid recovery from them.

2.2.2 DevOps benefits

Some advantages include:

- Fewer silos and improved communication among IT groups.
- Faster time to market for software.

- Rapid improvement based on feedback.
- Less downtime; improvement to the entire software delivery pipeline through builds, validation, and deployment.
- Less manual work due to automation.

We resume the values that Devops and its components add to the Agile project with Figure 2-3:

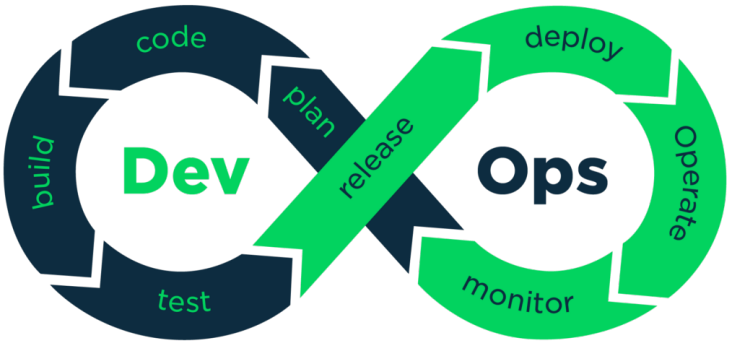


Figure 2.3: DevOps Process[10]

2.2.3 DevOps tools

Users may rely on CI/CD pipelines, containers, and cloud hosting in general. These tools might be open source, proprietary, or supported open source releases.

- **Code repositories:** Version-controlled source code repositories allow several developers to collaborate on code and, if necessary, revert to a prior version of code. Furthermore, tracking enables the monitoring of changes. GitHub and GitLab are only two of many options.
- **Artifact repositories:** For testing, source code is compiled into artifacts. Artifacts should also be gathered and stored in repositories such as JFrog.
- **CI/CD pipeline:** They allow for the validation and delivery of applications to end users via automation during the development lifecycle. The Continuous Integration tool takes off processes, allowing developers to build, test, and validate code in a common repository.
  Continuous Delivery expands on these automated stages by incorporating production-level testing and release configuration settings.
  Continuous Deployment goes a step further by triggering testing, configuration, and provisioning, as well as monitoring and potential rollback capabilities. Common tools are Jenkins, GitLab CI, CircleCI, and GitHub Actions.
- **Configuration Management:** Puppet, Chef, Ansible.
- **Monitoring:** Dynatrace, Prometheus. They are used to observe the performance and security of code releases.

2.3 GitOps

Weaveworks [11] debuted GitOps in 2017. It is a paradigm or collection of behaviors that was created primarily to manage Kubernetes clusters and deliver applications. In general, it is a cloud native paradigm that is closely tied to the continuous delivery method. GitOps emphasizes Git as a single source of truth. GitOps, as will be shown later, makes use of Git's various features and benefits.
It is a developer-centric strategy since developers are acquainted with Git and utilizing it to manage infrastructure simplifies everything. If the Git repository provides a declarative description of the necessary infrastructure, it can be referred to as an "infrastructure repository." GitOps embraces the IaC pattern, recasting it in a cloud-native environment and merging it with concepts such as orchestration, observability, declarativity, and immutable infrastructures. One of the goals of GitOps is to automate the deployment of what is included within the infrastructure repository.
However, it is also needed to automate the process of synchronization between the intended state (included within the Git repository) and the live state (deployed on k8s).

As a result, it is an operating model for k8s (or other cloud native technologies) that can be used to construct cloud native apps. GitOps not only provides best practices for containerized application deployment, administration, and monitoring, but also improves the developer's experience with application administration.

2.3.1 Principles

There are principles to follow if it is wanted to use GitOps:

- **Declarative approach:** Declarative configuration means that it is ensured by a collection of facts rather than a set of instructions. Everything must be declaratively described and kept in a Git repository. GitOps was intended to be used with k8s, which is mostly a declarative approach. As a result, it's a good thing GitOps follows the same technique.

- **Desired state versioned:** The intended state of the system must be kept in a Git repository. Because Git is a version control system, the infrastructure repository contains the infrastructure's history. Everything is derived and driven from a single location. It is simple to restore the infrastructure to a former condition.

- **Changes automatically applied:** The Git repository contains a description of the intended state. Every time it changes, the intended state changes as well. When a change occurs, the system automatically synchronizes with the new desired state. The most noteworthy aspect is that the user does not require cluster credentials to make that adjustment.

- **Software agents to ensure correctness and alert on divergence:** Once the system's state is stated and kept under version control, software agents can notify the user anytime the live state does not match the desired expectations, and they may eventually perform Continuous Deployment, synchronizing the live state. The usage of agents also assures that the system as a whole is self-healing.

Self-healing enhances the mechanisms that k8s already have.
The software agents serve as the operations' feedback and control loop.

2.3.2 Benefits

The benefits of applying GitOps best practices are:

- **Stored history of infrastructure changes:** All changes are logged since the application infrastructure can only be changed by making updates in the Git repository.
- **Productivity:** Continuous Deployment automation with an integrated feedback control loop speeds up the mean time to deployment. Development and operations teams can ship 30-100 times more changes per day, increasing overall development output 2-3 times. If the user needs a new environment or wants to update an existing one, he just has to modify the Git repository by pushing updates. There is no need to write scripts for Continuous Delivery.
- **Better Developer Experience:** For maintaining infrastructure, developers can use well-known programs such as Git. It is not necessary to know k8s in detail. Furthermore, any team member may check the repository and better understand the changes by using verbose commits and documentation published by other team members.
- **Stability:** Because it contains all cluster modifications, the Git repository may be utilized as an external audit log source.
- **Decoupling CI from CD:** GitOps technologies such as ArgoCD synchronize the infrastructure repository with the live state of Kubernetes (CD). These tools can be used independently with GitHub Actions, GitLab CI, and other (CI) tools without restriction.
- **Reliability:** The Git repository enables easy infrastructure rollback. Because the whole system is documented in Git, there is a single source of truth to recover from after a meltdown, cutting the mean time to recovery (MTTR) from hours to minutes.
- **Reduction of human error:** Declaratively, infrastructure is totally specified in YAML files hence repeatable and less prone to human error.
- **Approval process:** If the company uses a platform like GitHub or GitLab, it may be able to rely on the tools' approval mechanisms (pull requests). Some key branches, such as production, might benefit from a more fine-grained acceptance and review procedure. Other branches, such as feature branches, may be subject to fewer restrictions and hence be considered less significant.
- **Consistency and Standardization:** GitOps delivers an end-to-end process that is uniform and consistent throughout the entire enterprise.

- **Security:** Git's high accuracy and security guarantees, backed up by the strong encryption used to monitor and manage changes, as well as the ability to sign changes to establish authorship and origin, are critical for a safe specification of the cluster's intended state.

2.3.3 Tools

GitOps allows for the use of many technologies to construct the CI/CD pipeline; there are no hard and fast rules. Tools that are open source or proprietary can be used. The three primary GitOps native technologies that allow continuous delivery are as follows:

- **ArgoCD**
- **Flux**
- **Jenkins X**

It is critical to recognize that GitOps provides a solution for Continuous Delivery or Deployment. Other technologies, such as Jenkins, GitLabCI, and Github Actions, must surround the GitOps CD tool in order to construct a complete solution that takes into account Continuous Integration. A k8s controller that implements the operator pattern may be used to listen for and coordinate deployments to the k8s cluster (e.g. ArgoCD). This method is more secure and automates the time-consuming process of updating YAML files. It enables Continuous Deployment because when the desired state (Git repository) and the live state diverge, the operator synchronizes the live state.

2.3.4 Push-based pipeline

A push-based pipeline [12] means that code begins with the Continuous Integration steps and then proceeds through a series of scripts or manually utilizes kubectl to push changes to the k8s cluster. This is currently the typical CI/CD method.

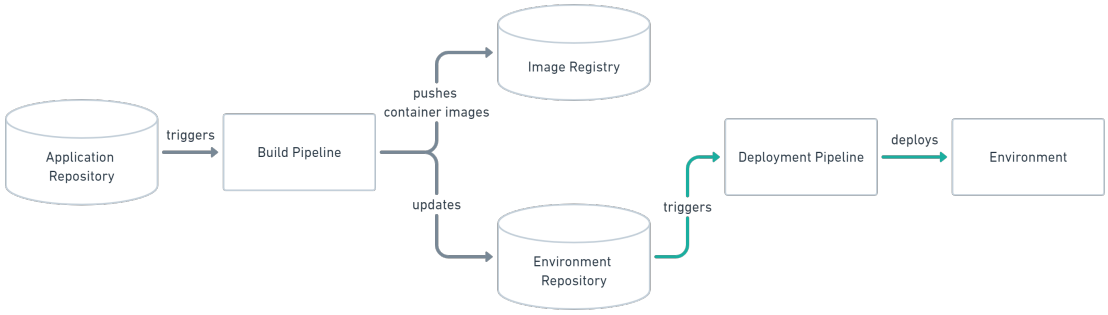


Figure 2.4: Push-based pipeline [13]

Because the deployment is not declarative, this is not the best approach. Furthermore, the CI/CD pipeline must include the k8s cluster credential, which is a risk. Finally, the

downside of this strategy is that the live state does not immediately synchronize with the intended one because there is no k8s operator such as ArgoCD or Flux.

2.3.5 Pull-based pipeline

The pull-based pipeline [12] uses the same CI implementation as the pull-based pipeline, but the way continuous deployment is performed changes. An operator, such as ArgoCD, is presented here. It assumes the function of the pipeline by comparing the desired state in the infrastructure repository to the live state in the k8s cluster continually. When discrepancies are discovered, the operator adjusts the infrastructure to match the current status. Furthermore, the image registry may be watched to detect new versions and update them. This option, like the push-based deployment, updates the environment anytime the environment repository changes. Furthermore, using the operator, changes may be seen in different ways. If the live environment delivered changes in any way that is not defined in the infrastructure repository, these modifications are undone. By making all direct modifications to the cluster impossible, this guarantees that all changes are traceable in the Git log. This shift in strategy addresses the issue of push-based deployments, in which the environment is only updated when the infrastructure repository changes. The k8s cluster credential is not sent to the pipeline because the operator compares the live state to the intended one.

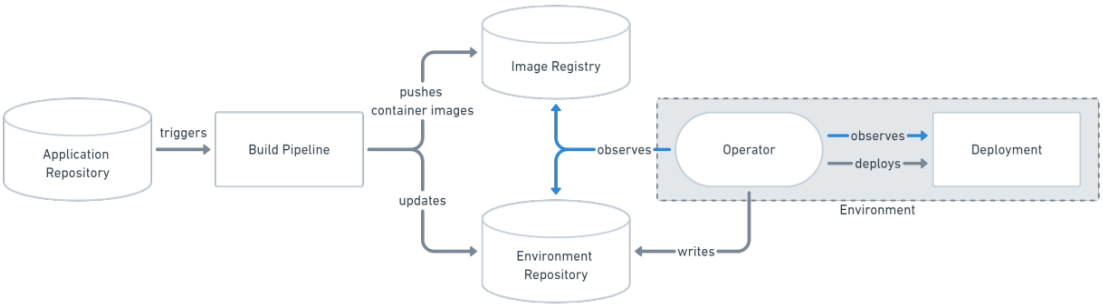


Figure 2.5: Pull-based pipeline[14]

2.4 Virtualisation

Virtualization [15] is used to simulate physical machines in the form of VMs or containers. It is a representation of IT components such as servers, storage, and networks that are hardware into a virtual equivalent that can be provided as a software. Virtualization is the modern way to reduce IT costs while enhancing efficiency and agility for all size businesses. We have mainly two different kinds of virtualization that we need to study through this project:

- **Paravirtualization:** A virtualization method that provides a software interface to VMs that is similar but not identical to that of the underlying hardware.

- **Containerization:** Instead of running an application on a VM, containerization consists of running it in a fully packaged and portable computing environment called containers which are abstracted away from the host operating system.

To a better understanding of virtualisation, we will list the differences between containers and VMs on Figure 2-6 :

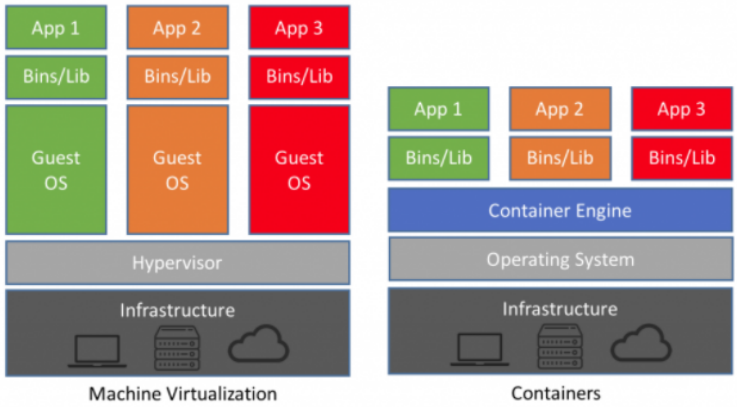


Figure 2.6: VMs vs Container [16]

A VM runs over a hypervisor (or VM monitor) which provisions its resources. On the other side, containers are managed through a "Container Engine". The most known engine nowadays is Docker. Moreover, a VM runs on top of its own operating system. While a container necessitates an underlying operating system that provides the basic services to all the containerized applications.

2.5 Microservice architecture

A microservices architecture [17] is an approach that takes loosely coupled services which and modifies them to be developed, deployed, and maintained independently. Each one of these services handles a distinct task and they can all communicate through simple Application Programming Interfaces (APIs) to solve a larger complex business problem. The difference between the microservices and monolithic architectures is presented through Figure 2.7.

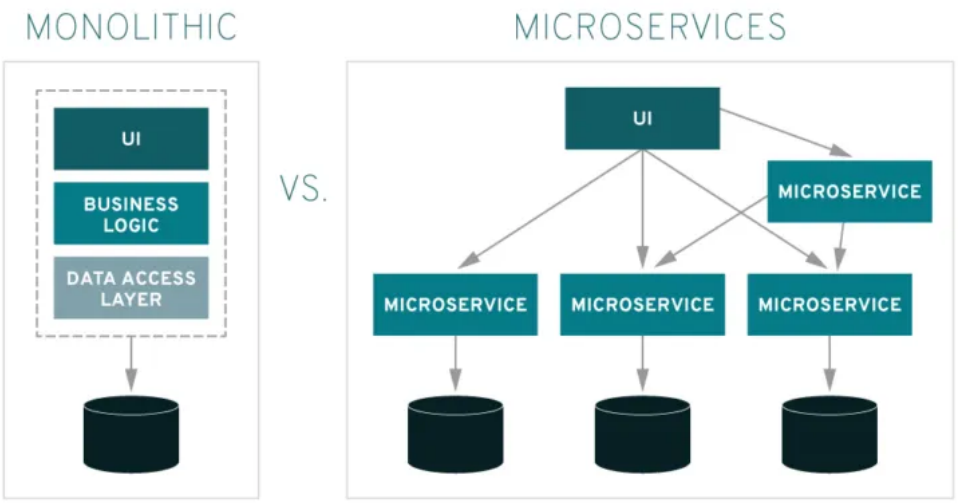


Figure 2.7: Microservices vs Monolithic architecture [18]

Microservice has too many benefits that made leader-worldwide companies implement them in their projects. From these benefits we can mention:

- Failisolation: A failing service will not affect the whole project.
- Makes update easier: With the simplicity that it implements, developers can better understand the functionality of a service.
- Scalability: Separated services can be easily scaled.
- Smaller and faster deployments: Working on each service separately makes it easier to update modules through CD.

2.6 Monitoring

IT monitoring [19] is the method of gathering metrics about the hardware and software processes of an IT environment to ensure that everything works as intended to help applications and services. Basic monitoring consists of system activity scans, while advanced monitoring provides granular views of operating statuses such as average response times, number of application instances, error and request rates, CPU utilization, and application availability. IT monitoring covers three sections:

- **Foundation:** Infrastructure refers to the physical or computational components that make up a computing stack, such as servers, CPUs, and virtual machines.
- **Software:** This section, also known as the monitoring section, examines what is happening on the foundation's computers, such as CPU consumption, load, memory, and a running VM count.
- **Interpretation:** Metrics are shown in graphs or data tables, often on a graphical user interface (GUI) dashboard. This is often achieved by combining data visualization techniques with other software.

Nowadays, enterprises equip their projects with monitoring platforms to ensure their high availability, and that's for their numerous benefits. Among these advantages we mention:

- **Early detection of problems:** regular checks make it so easy and sure to detect problems at an early stage.
- **Avoid outages:** Keeping an eye on the performance of the infrastructure helps to prevent all kinds of resources outage such as network, CPU, Memory outage.
- **Fix IT performance issues quickly:** No need to scan the whole project to find the reason of the issue because the monitoring platform will lead directly to it.
- **Recognise security threats:** Regular routine checks reveal different security issues such as unauthorized accesses, forbidden ports…
- **Improve reliability of IT resources:** Reliability is one of the major factors to assure the project's quality. Monitoring boosts the project's reliability by assuring its high availability and its quick recovery in case of problems.

To ensure the reliability and the high availability of our project, we will implement a monitoring platform using Prometheus to collect metrics of our kubernetes cluster and visualize them using Grafana and finally sending alerts with AlertManager.

2.7 Conclusion

Through this chapter, we have clarified the different concepts that will help us to better undertake our project. The next chapter will cover the details of the specifications of the needs along with a planification that will help us to implement our solution.

Chapter 3

Analysis and Conception

Introduction

After presenting the general framework of our project, the concepts and the key elements, we will pass through this chapter to a crucial phase in the development cycle of our solution which will allow us to identify the actors, formalize the expected needs, and determine the functionality of the system. Thus, we will present in the following a study of functional and non-functional needs and the global use case diagram. Finally, we will present the proposed roadmap, the working environment and the automation solution.

3.1 Specification of needs

In this section, we start the specification of the requirements that must meet our system. The purpose of this phase is to identify the actors and describe the different functional and non-functional needs.

3.1.1 Actors

The analysis part always begins with the determination of the different actors in carrying out a study of the interaction of our system with its environment. In the frame of our project, users are classified according to the functions granted to them. In this project context, we can identify one main actor interacting with the system in different ways:

- The integration engineer.

3.1.2 Functional needs

The second step in the design process is to analyze the situation to take into consideration the constraints, risks and any relevant elements. We will therefore present the functional and non-functional requirements. These Functional needs of our system are:

- Automation of 'kubernetes' deployment.
- Deployment of ArgoCD.
- Using Helm Chart to write the manifests files.
- Deployment of the application on the Kubernetes platform
- Administration and supervision of our kubernetes cluster.

The Figure 3-1 presents Global use case diagram:

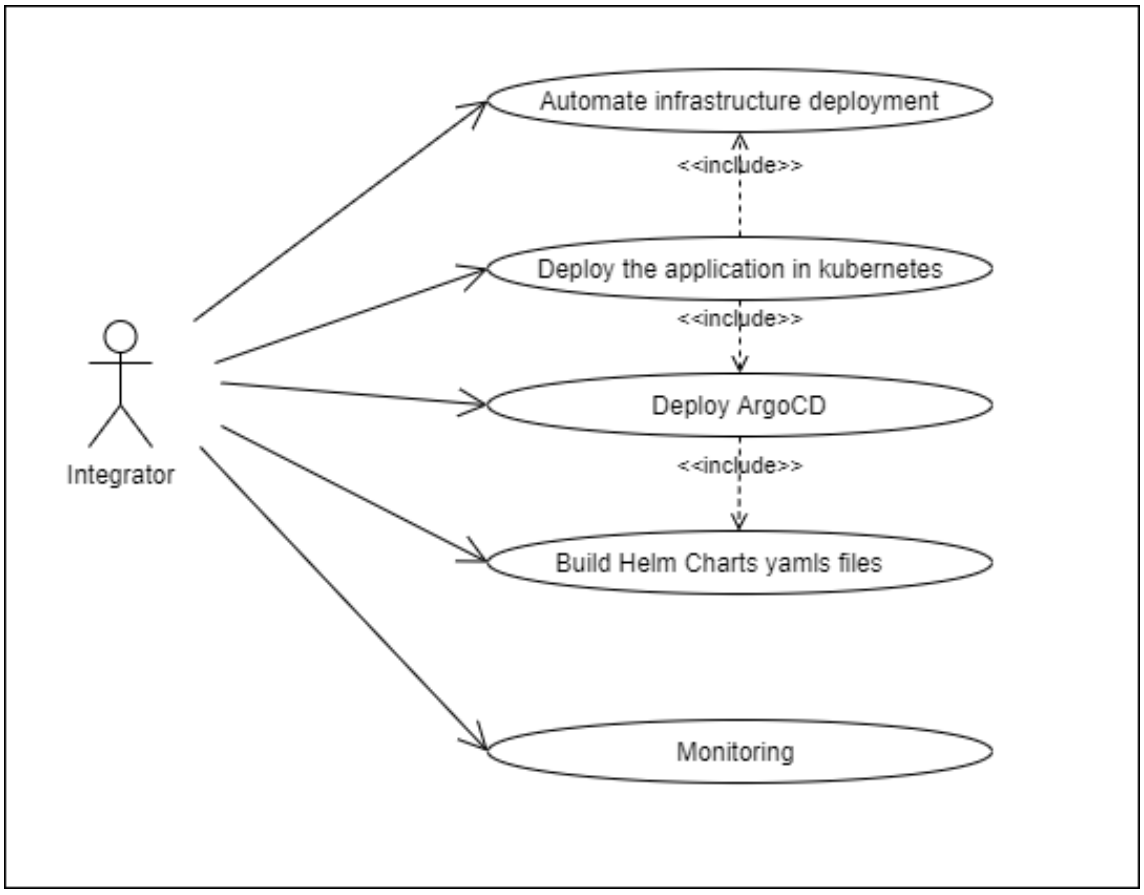


Figure 3.1: Global use case diagram

3.1.3 Non-functional needs

In addition to functional requirements, there are several non-functional restrictions that must be adhered to in order to ensure the system's qualifying property and enable us to construct a high-performance solution. Among these requirements we distinguish:

- **Time:** The various tools must have the shortest possible response time.
- **Simplicity:** The user must have easy access to the resources, thus allowing him to accomplish his objectives with great certainty.
- **Scalability:** The system must be able to support the evolution and scalability of its components, and the ability to add nodes as the load increases.

3.1.4 Proposed architecture

To better explain this scenario, we have constructed a global architecture to our solution, as shown by the Figure below:

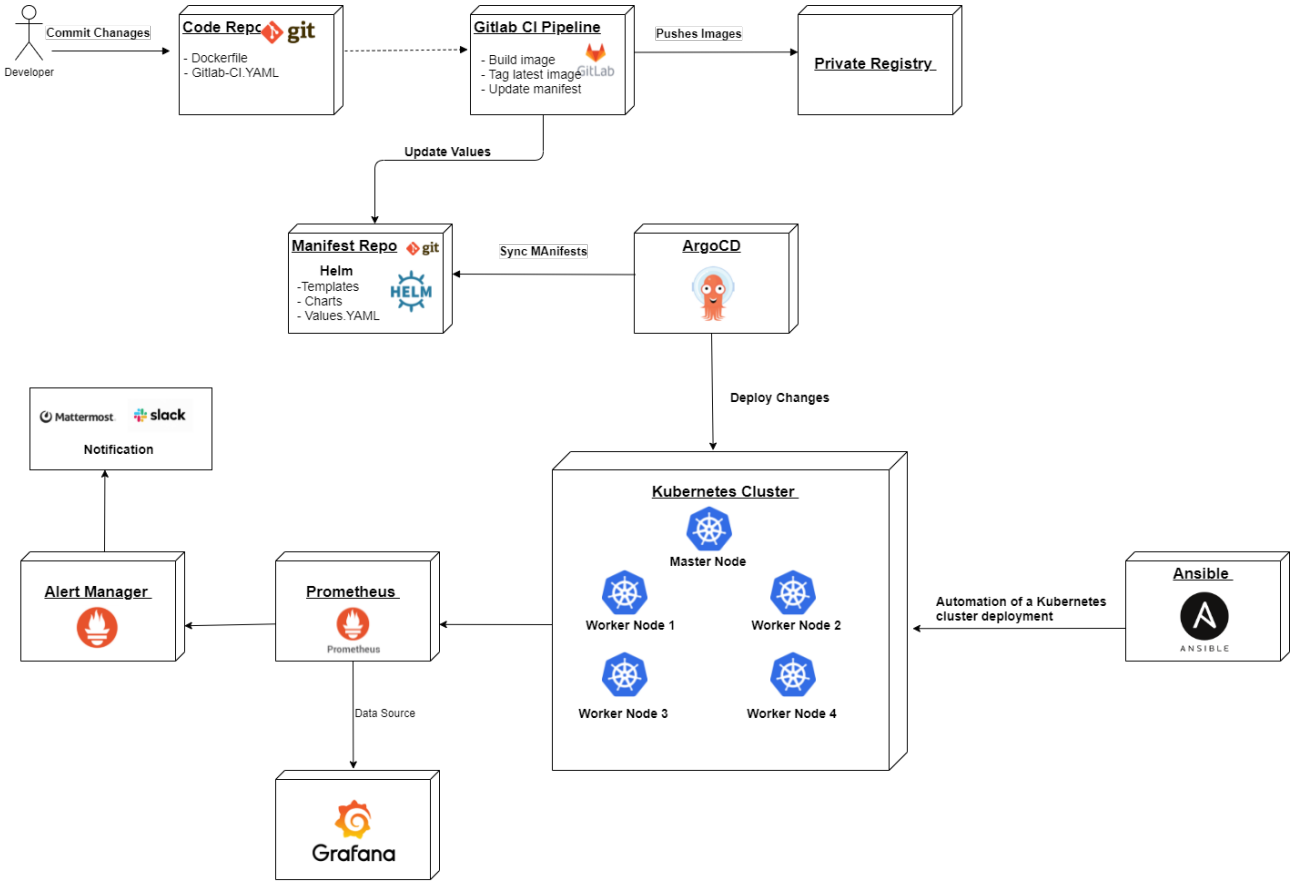


Figure 3.2: Global architecture

Although we have a CI pipeline ready, we need to make modifications at the different stages of the pipeline, such as changing the data source url in the application properties of the backend of our application reference and the different IP addresses we will need.
The second phase will be the automation of a Kubernetes cluster deployment using Ansible. (Kubernetes is detailed in Appendix 6.3)
Then we deploy ArgoCD and create the update of the manifests files in the git repo so that ArgoCD can synchronize the manifests and deploy the application on the kubernetes cluster.
Then we end implementing Prometheus , AlertManager and Grafana with Kubernetes to monitor our cluster.

3.2 Working environment

In this section, we will present the hardware and software environment of the project.

3.2.1 Hardware environment

To accomplish this project, we have used the enterprise's personal custom computer for development and the enterprise's virtual machines to deploy the infrastructure and the project. The personal computer has the following configuration:

- CPU: Intel Core i5-4210U 2.40 GHz
- RAM: 12 GB
- Storage: 500 GB

Each virtual machine has the following configuration:

- OS: CentOs 7
- CPU: Intel(R) Xeon(R) CPU E5-2420 0 @ 1.90GHz
- RAM: 4GB
- Storage: 50 GB

3.2.2 Software environment

For the software environment, we have worked with several technologies and libraries:

Containerization

Docker

Docker [21] Containerization engine used to build, run, and manage containerized applications. Containers may be treated as highly lightweight, modular virtual machines using Docker technology. Furthermore, we have a lot of flexibility with these containers since we can create, deploy, copy, and transfer them from one environment to another, allowing us to optimize our apps for the cloud.

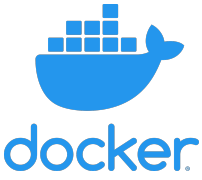


Figure 3.3: Dockers logo[20]

Version-Control System

Orange Gitlab

Gitlab is a project hosting and management platform. It is described as a platform for modern developers, and it allows to manage Git repositories and therefore better comprehend the administration of the source code versions.



Figure 3.4: Gitlabs' logo[22]

Orchestration

Kubernetes

An open-source orchestrator used for managing multi containers on a large scale. It eliminates many manual processes associated with the deployment and scaling of containerized applications.



Figure 3.5: Kubernetes' logo[23]

CI/CD

Gitlab CI

Gitlab-CI [25]: A Gitlab service that builds and tests whenever the code repository is updated through a pipeline that can be defined programmatically through a custom file.



Figure 3.6: GitlabCIs' logo[24]

ArgoCD

Argo CD [27] is a declarative, GitOps continuous delivery tool for Kubernetes. It follows the GitOps pattern of identifying the target application state using Git repositories as the source of truth. Argo is detailed in Appendix A.2.



Figure 3.7: ArgoCD's logo[26]

Monitoring

K8S Lens

K8S Lens [29] is a desktop client for managing Kubernetes clusters, that was initiated in 2020. It is an open-source Dashboard for controlling and administrating Kubernetes Clusters.



Figure 3.8: Lens' logo[28]

Grafana

Grafana [31] is an open-source tool for doing data analytics, retrieving metrics that make sense of large amounts of



Figure 3.9: Grafana logo

data, and monitoring our programs using cool configurable dashboards.

Prometheus

Prometheus is a free and open-source monitoring and alerting program for computers. It keeps track of real-time measurements in a time-series database depending on the content of HTTP-enabled entry points.



Figure 3.10: Prometheus logo [32]

AlertManager

Alertmanager [34] is a Prometheus component that allows you to set and manage Prometheus server alerts, as well as route them to notification, paging, and automation systems.



Figure 3.11: AlertManager logo[33]

Management Configuration Tool

Helm Chart

Helm [36] uses a chart-based packaging system. A chart is a group of files that represent a set of Kubernetes resources that are linked. A single chart may be used to deploy anything as basic as a memcached pod or as complex as a complete web app stack of HTTP servers, databases, caches, and other components.



Figure 3.12: Helm logo [35]

Ingress

Nginx Ingress Controller

NGINX Ingress Controller [38] is a best-in-class traffic management solution for cloud-native apps in Kubernetes and containerized environments.



Figure 3.13: Nginx's logo[37]

Automation

Ansible

Ansible [40] is an opensource tool to automate the provisioning and configuration management of apps and IT infrastructure. It is used for application deployment, server and workstation upgrades, configuration management, intra-service orchestration, and other activities that a system administrator performs on a daily basis.



Figure 3.14: Ansible logo [39]

Data Base

MariaDB Data base

MariaDB [42] Server is a well-known open source relational database. It was created by the initial MySQL creators and is guaranteed to remain open source. It is used in most cloud services and is the norm in most Linux distributions. MariaDB is the used database used with the applications deployed.



Figure 3.15: MariaDB logo [41]

Data Persistence

OpenEBS Driver

OpenEBS Driver [43] is a tool to manage Container-Attached Storage and attaching dynamic PVs and PVCs.



Figure 3.16: OpenEBS logo [40]

Programming Environment

Visual Studio Code

Visual Studio Code [45] is a simplified code editor that supports development tasks such as debugging, task execu-



Figure 3.17: Visual Studio Code logo [44]

tion, and version control.

Bash Scripting

Bash Scripting [**bash**]: A computer programming language designed to run by the Unix Shell to run different commands that will be used to automate a few system routine instructions.



Figure 3.18: Bash Scripting logo [46]

Modeling and illustration

Draw.io

Draw.io is a free online diagram program that can be used to create flowcharts, process diagrams, org charts, UML, ER, and network diagrams.



Figure 3.19: Draw io logo [47]

Microsoft PowerPoint

Microsoft PowerPoint: A tool that allows you to create and show slides to support a presentation. It can also combine text, graphics, and multi-media to build animated illustrations.



Figure 3.20: Microsoft PowerPoint logo[48]

3.3 Conclusion

In this chapter, we presented the general planification phase of the project that puts us on the right path of the implementation of our project through the exportation of the needs. We also specified the working environment. We will pass in the next chapters to present the realization of each step of our work.

Chapter 4

Kubernetes Infrastructure Automation

Introduction

In this chapter, we will explore the first step which is the Kubernetes infrastructure deployment automation. We will first present a detailed analysis of the steps through different diagrams. Then, we will pass to their realization where we will explain the different steps that we have passed through to reach the final result.

4.1 Use case diagram

To be able to implement our solution, we have to set up an eligible infrastructure that could run Kubernetes and other environment factors. To do so, we have to work with the virtual machines of Sofrecom to host kubernetes. Since this process can be so complex on the VMs, we will automate the whole operation so the cluster can be created via simple steps.

The Figure 4-1 describes the refined use case of automate infrastructure deployment:

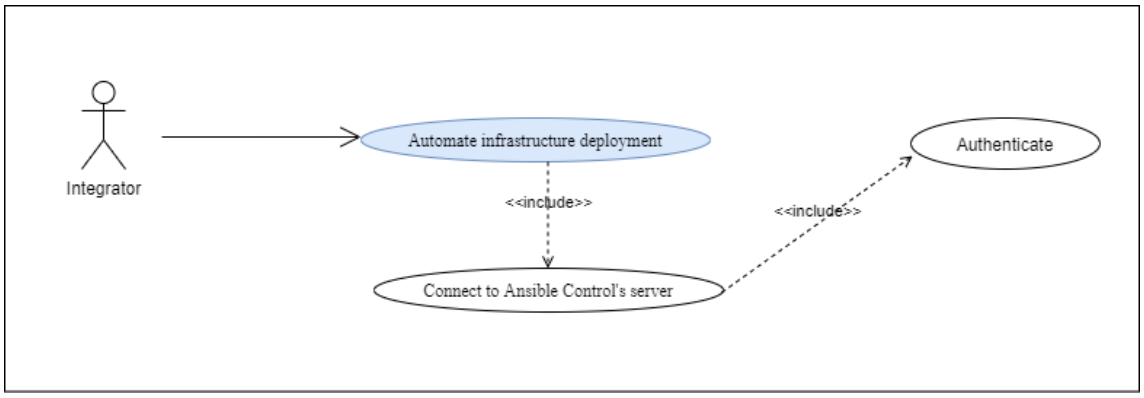


Figure 4.1: Automate infrastructure deployment use case

The Table 4-1 represents the description of the use case “automate infrastructure deployment”:

Table 4.2: Description of automate infrastructure deployment use case

Title	**Automate Infrastructure Deployment**
Actor	Integrator
Review	A use case that allows the actor to deploy a fully working Kubernetes cluster on the VMs
Pre-condition	Infrastructure VMs created and initiated
Post-condition	Kubernetes cluster fully deployed and ready to run the project

Nominal Scenario	* The Actor connects to the control server and inserts the infrastructure's VMs information. * The Actors launches the deployment script from the Control Server.
Error Scenario	E1: in step 2: The process can fail if wrong credentials are inserted: An Error message would be showing the issuing VM and the error.

4.2 Sequence diagram

To better understand the processes described in the Table 4-1, we will pass to the system's minimal sequence diagram, which details the nominal scenario of the use case "automate infrastructure deployment ". The Figure 4-2 presents the sequence diagram:

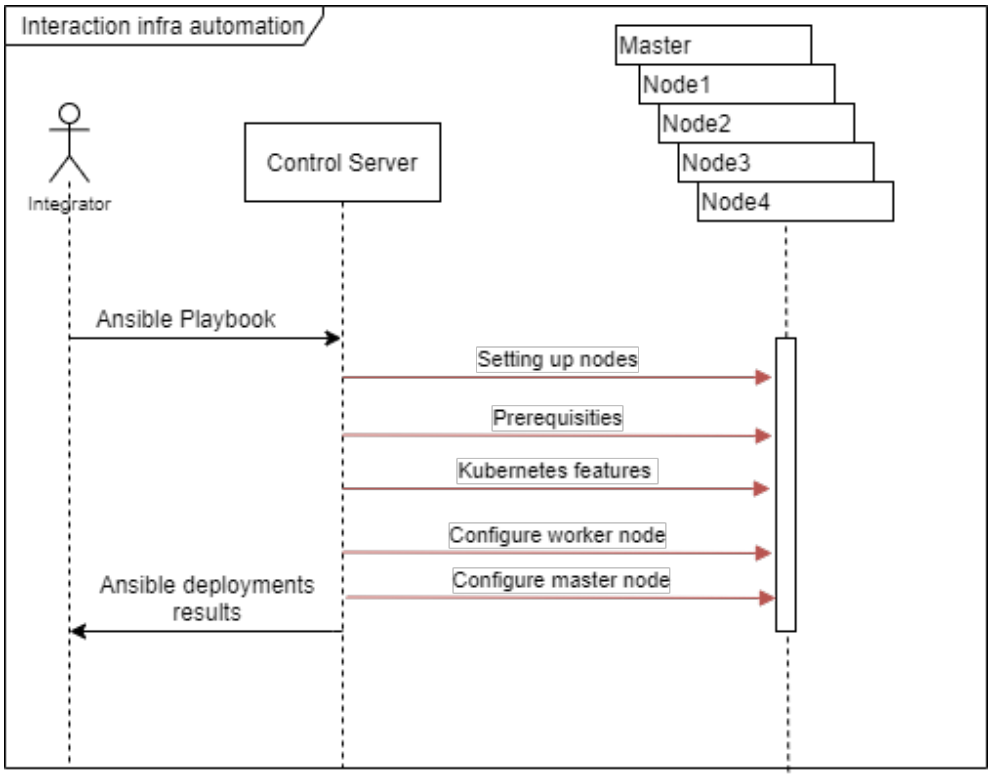


Figure 4.2: Automate infrastructure deployment sequences diagram

The VMs' details should be added to the control server to inform it about the VMs that construct the cluster, their IPs, how it will connect to them and the role of each one of them. Once the control server is set up, the integrator executes the automation script that will automatically launch all the instructions from the ansible playbooks until the cluster ends fully deployed. At the end of this process, the control server will show an output message to tell if the deployment has ended successfully or aborted because of an error with specifying the reason and its source VM.

4.3 Realization

In this section, we will present the different steps of the realisation of this step.

4.3.1 Deploying the infrastructure manually

Before passing the automation part, we must first configure the infrastructure manually and note the necessary steps to deploy Kubernetes.

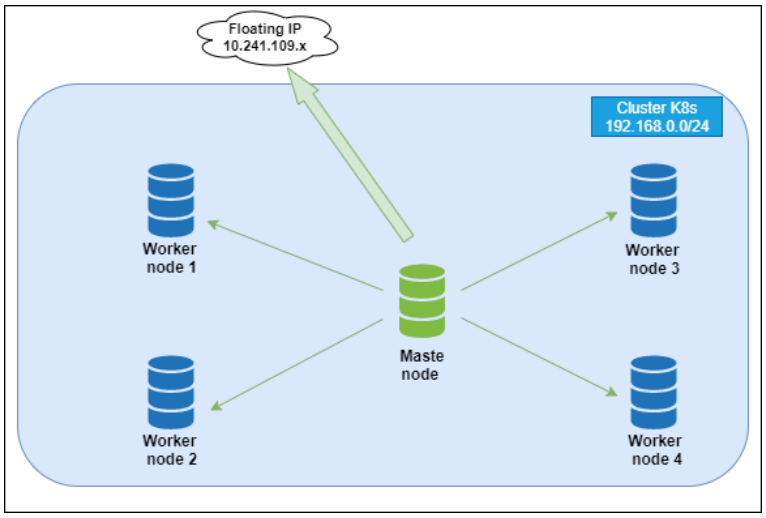


Figure 4.3: Infrastructure initial architecture

Through this step, we can distinguish the different steps to install Kubernetes. We have to access each machine separately several times with its own floating IP.

4.3.2 Automating the deployment

To automate the Deployment, we have created the following Ansible project presented in Figure 4-4.

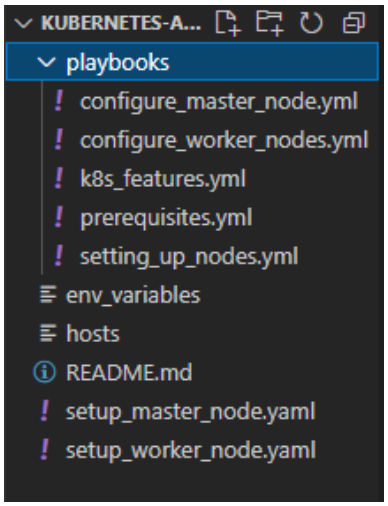


Figure 4.4: Kubernetes Ansible project content

The Kubernetes ansible project includes the following components

- playbooks: contains the functions iterated by the project.
- hosts: defines cluster hosts.
- env-variables: define environment variables such as cluster IPs, kubernetes version…

In addition, we have created another file in the playbook 'k8s-features.yaml" to automate some Kubernetes functions such as the deployment of OpenEBS Storage Driver, CertManager and Ingress Nginx Controller.

After setting the environment and starting the playbooks, we will get a working fully functionating infrastructure running Kubernetes with a master and four worker nodes. The figure below represent the adding of the worker-node3 and worker-node4.

```
PLAY RECAP *********************************************************************
*
worker-node3               : ok=21   changed=10   unreachable=0    failed=0
skipped=0    rescued=0    ignored=0
worker-node4               : ok=21   changed=10   unreachable=0    failed=0
skipped=0    rescued=0    ignored=0

ansible@ansible:/etc/ansible/kubernetes-ansible$
```

Figure 4.5: Kubernetes Ansible project output of the adding of the worker nodes

The new architecture that we get after the automation phase is presented in Figure 4-6:

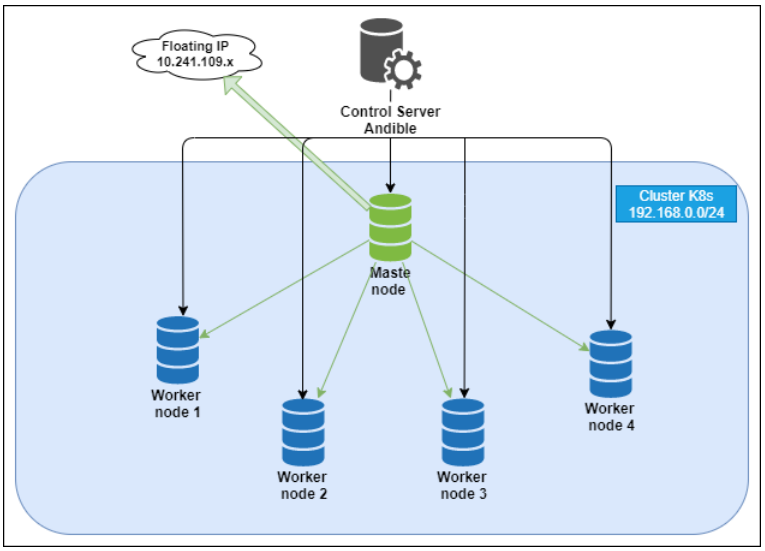


Figure 4.6: Infrastructure automated architecture

Usually, the Kubernetes dashboard is provided via the kube proxy, giving access only while running the proxy and connected to the cluster nodes. In our case, the first thing that we have configured is the dashboard deployment. The dashboard has been deployed on an independent namespace exposing two services as shown below in this figure:

```
[root@master-node ~]# kubectl get pods -o wide --namespace kubernetes-dashboard
NAME                                         READY   STATUS    RESTARTS   AGE    IP              NODE           NOMINATED
NODE   READINESS GATES
dashboard-metrics-scraper-7b59f7d4df-6fhzz   1/1     Running   0          3d1h   192.168.246.6   worker-node1   <none>
       <none>
kubernetes-dashboard-74d688b6bc-4fn4z        1/1     Running   0          3d1h   192.168.240.6   worker-node2   <none>
       <none>
[root@master-node ~]# kubectl get svc -o wide --namespace kubernetes-dashboard
NAME                        TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)         AGE    SELECTOR
dashboard-metrics-scraper   ClusterIP   10.109.170.200   <none>        8000/TCP        3d1h   k8s-app=dashboard-metrics-sc
raper
kubernetes-dashboard        NodePort    10.96.121.55     <none>        443:32444/TCP   3d1h   k8s-app=kubernetes-dashboard
[root@master-node ~]#
```

Figure 4.7: Dashboard added namespace

The kubernetes-dashboard has been deployed with a NodePort service. This NodePort exposes the Service on each Node's IP at a static port, using the metrics provided by the dashboard-metrics-scraper (the second ClusterIP service) on each node. The dashboard is ready, and the access is granted through the associated port after setting the dashboard-admin-user.
Other than the kubernetes way of exposing the services, we aimed to install and configure an external Ingress controller by Nginx.
Ingress is most useful when exposing multiple services under the same IP address, and these services all use the same L7 protocol.
Nginx Ingress Controller was deployed on a new namespace ingress-nginx.
The deployment of the ingress controller as a DaemonSet on all schedulable nodes, Ports will be exposed on each node where the controller is deployed as shown in the figure below so we could access the kubernetes dashboard using our DNS apps-olt.sofrecom-tunisie.com :

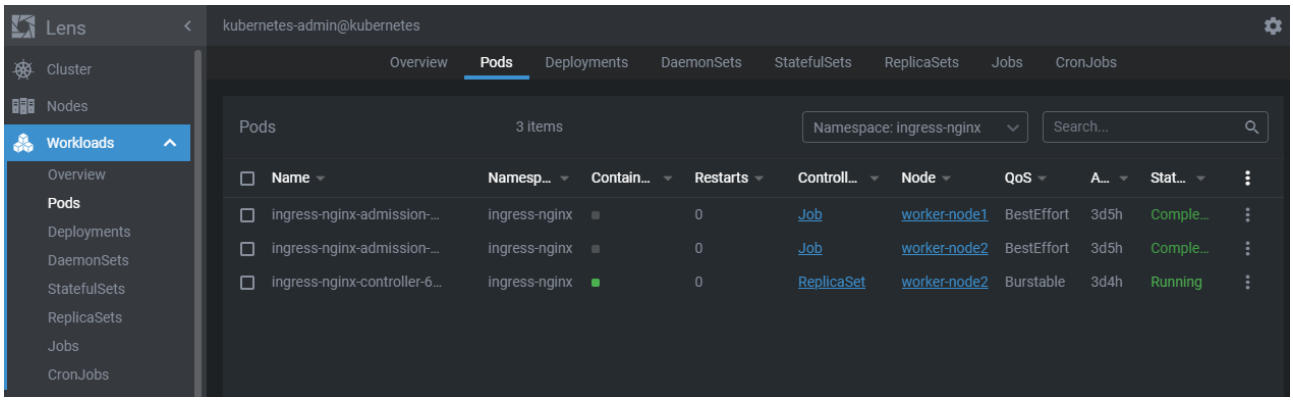


Figure 4.8: Nginx deployment on Kubernetes

As shown on the dashboard in figure 4.8, the Nginx Ingress Controller has been deployed on different worker nodes. If we add a new node to our cluster, the controller will be deployed automatically.

As shown below, a kube-system namespace is added, containing a set of services that administrate and orchestrate the whole cluster. The pods are replicated and scaled to be deployed on different nodes.

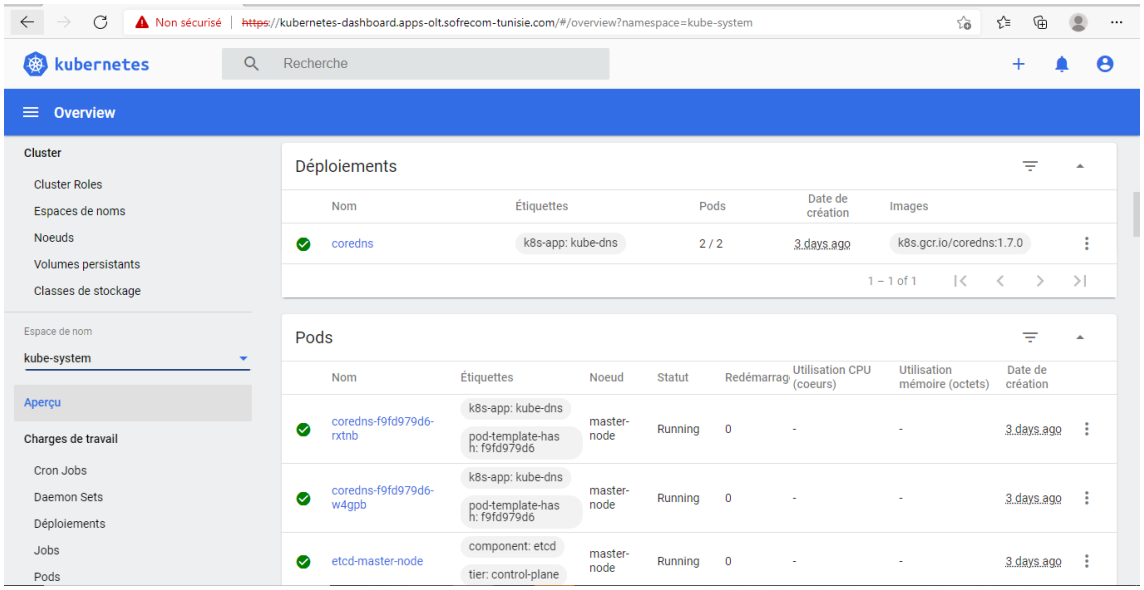


Figure 4.9: Kubernetes dashboard

4.4 Conclusion

In this step, we deployed a highly available Kubernetes infrastructure that is ready to host the applications. In the next step, we will deploy ArgoCD to deploy the wassup application on kubernetes.

Chapter 5

Orchestrations

Introduction

In this chapter, we will present the second step, which is the deployment of the application. We will first start by creating the manifest files with Helm Chart, then w will pass to set a CD pipeline using ArgoCD to deploy the application on our Kubernetes cluster and automate the process.

5.1 Kubernetes Resources for Deployment

With our application containerised, we move to the step which is to deploy it on the Kubernetes infrastructure. To achieve this, we have to write the YAML files for defining the necessary resources. We detail those below.

- **Deployment:** Deployments describe the desired state for our app (image to deploy, number of replicas, resource limits, health checks, etc.).
- **Service:** The service endpoint routes traffic to one or many instances of our application from a single internal IP address. It uses the labels created in the deployments as selectors to identify the application.
- **Ingress:** The IP addresses used by the service are internal. They can only be used inside the cluster. Ingress exposes HTTP and HTTPS endpoints so that traffic from outside the cluster can be routed to services within it.

5.2 Templates for Deployment Files: HELM

By default, Kubernetes resource files are static. There is no way to add variables or avoid repetition and streamline modifications. This inspired the community to build a simple yet powerful tool that allows the overcoming of these challenges. The tool was called HELM.

Helm[49] works with entities called Charts. These charts help define, install and upgrade even the most complex Kubernetes applications. Helm alleviates the users from having to write repetitive yaml, provides consistency between local and live states, makes intelligent deployments by determining the order that Kubernetes deployments need to be created (e.g. cannot create Ingress before the Deployment and Service are ready). Helm can also manage the life-cycle of a deployed application (install, update, rollback, uninstall).

Helm uses templates to eliminate having to copy and paste resource definitions and having minor changes added to them as shown in figure 5.1. The values in red can be put in separate Values.yaml while the rest of the file remains a static template. When it is time to deploy, we simply generate a working yaml file using the template and the correct values from the Values.yaml file and Helm will deploy the resources with the correct values on the Kubernetes cluster.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-k8s-app
spec:
 replicas: 1
 selector:
  matchLabels:
   app:my-k8s-app
 template:
  metadata:
   labels:
    app: my-k8s-app
  spec:
   containers:
    - image: my-k8s-app:v1
     imagePullPolicy: IfNotPresent
     name: app
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-k8s-app
spec:
 replicas: 1
 selector:
  matchLabels:
   app:my-k8s-app
 template:
  metadata:
   labels:
    app: my-k8s-app
  spec:
   containers:
    - image: my-k8s-app:v3
     imagePullPolicy: IfNotPresent
     name: app
```

Figure 5.1: Example of Repetitive YAML resources



Figure 5.2: Faster and Smarter Deployments with HELM and ArgoCD

When paired with ArgoCD, HELM can make deployments happen at the push of a button.

After configuring our Helm charts like is shown in the Figure below, we pass to the configuration of ArgoCD to deploy our application.

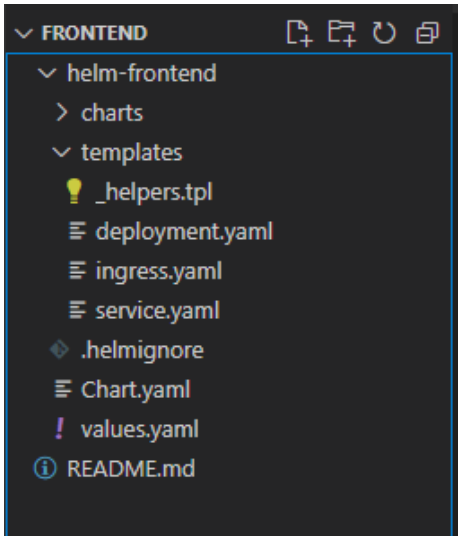


Figure 5.3: Frontend Helm Chart

5.3 ArgoCD installations and configurations

In this section, we present the installations and configurations of ArgoCD for the deployment of our application.

5.3.1 Installation

First, we have created a new namespace "ArgoCD" within our cluster where Argo CD services and application resources will live. Then we have downloaded the latest Argo CD version from Github and deployed it using the command "kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml". Now we have all pods running within our cluster, as shown in the figure below.

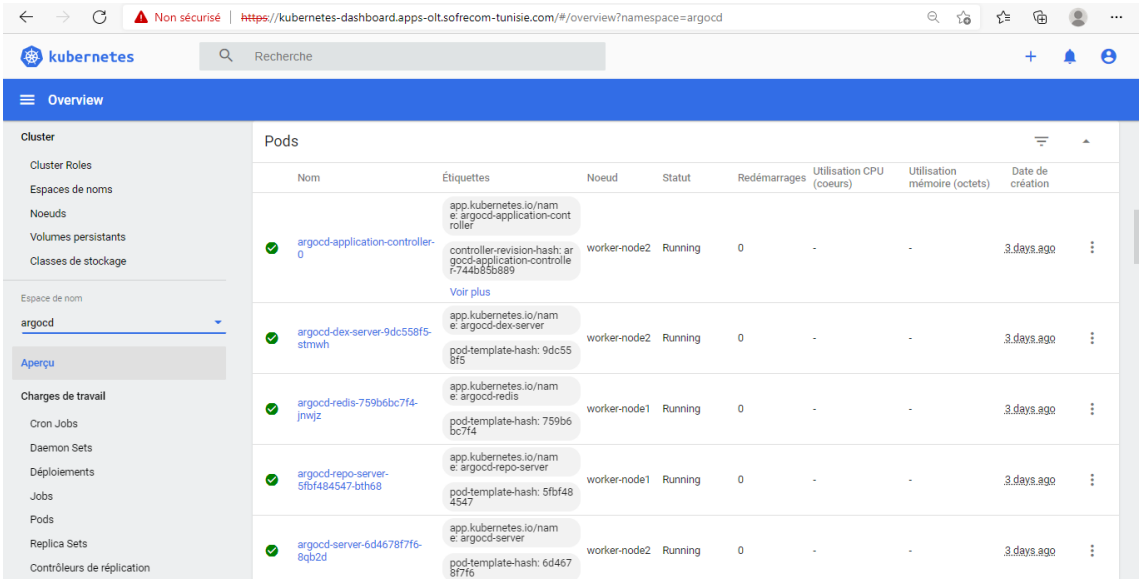


Figure 5.4: ArgoCD's pods running

5.3.2 Configuration

By default, the Argo CD API server is not exposed to an external IP, so we used Nginx Ingress Controller technique to expose it as we used with Kubernetes dashboard. Also, we have assigned a TLS (Transport Layer Security) certificate for argocd-server since ArgoCD runs a HTTPS server.

After exposing the ArgoCD API server, we have logged in to the Argo dashboard like is shown in the figure below.

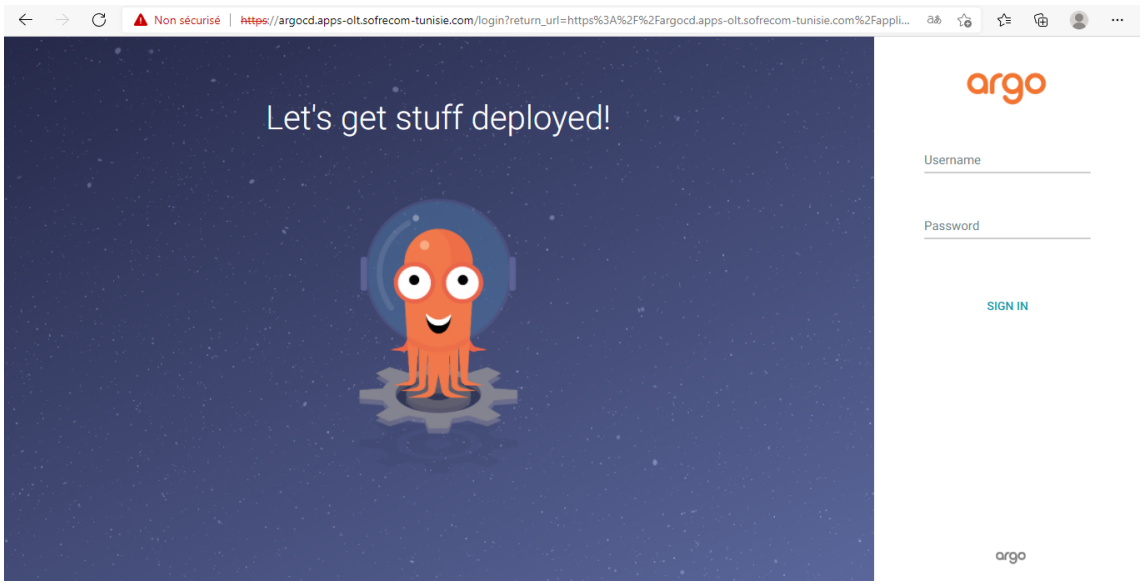


Figure 5.5: ArgoCD's Dashboard

Note that the same work will be repeated three time for each deployment of the WASSUP application, the frontend, backend and the database. We will present the frontend deployment only as an example.
To deploy our frontend, we must first set up the Git code repository that will contain all of the YAML files or Helm chart templates that will be used as shown in the Figure 5.6 below.

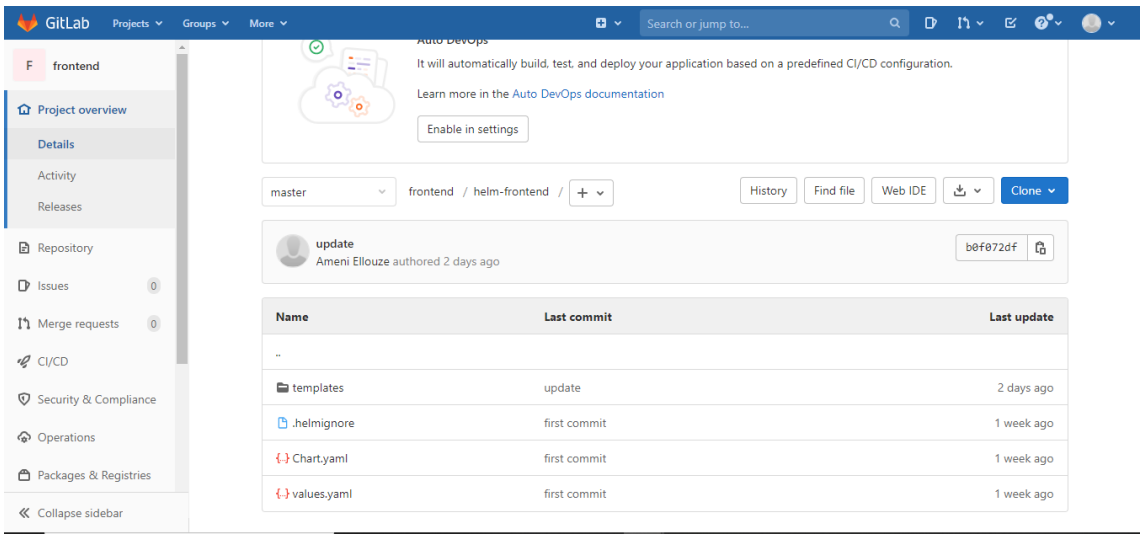


Figure 5.6: Frontend Helm Gitlab repository

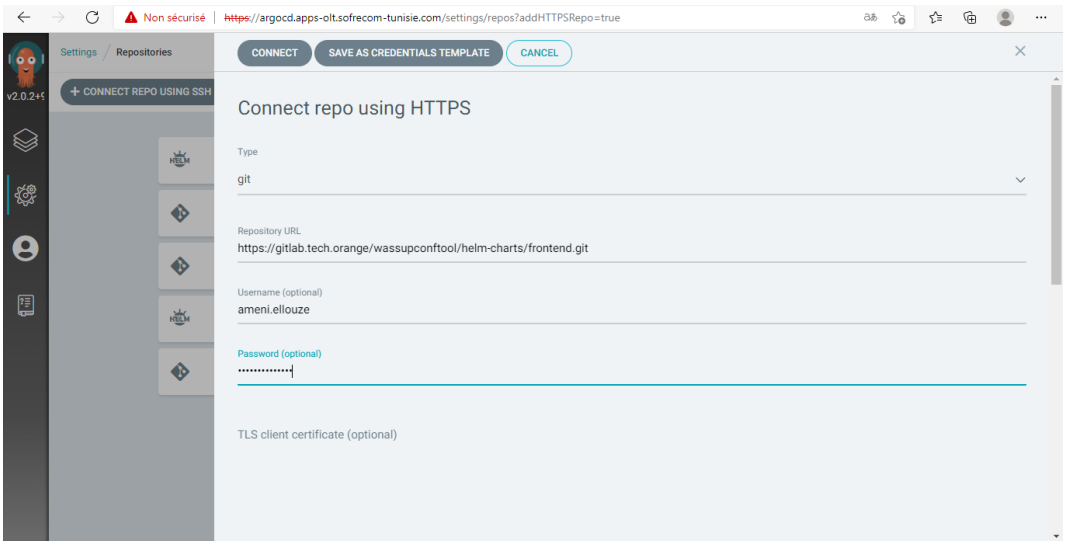


Figure 5.7: Connect repo using HTTPS

Once the repository of our code connected, we created an application by following the menu Manage your applications, and diagnose health problems > New app.

A pane opened with several categories:
General: Here we have configured the standard parameters of the application. The Sync policy allows us to define whether we want our application to synchronize automatically or manually.
Source: The code repository is configured with the Revision, which allows us to select a specific branch, tag or commit.
Destination: This section is used to specify the cluster on which we will deploy our application. By default, the cluster where Argo CD is installed is set.
In our case, we used Helm Chart from which an additional section is displayed as shown in the Figure 5.8 below:

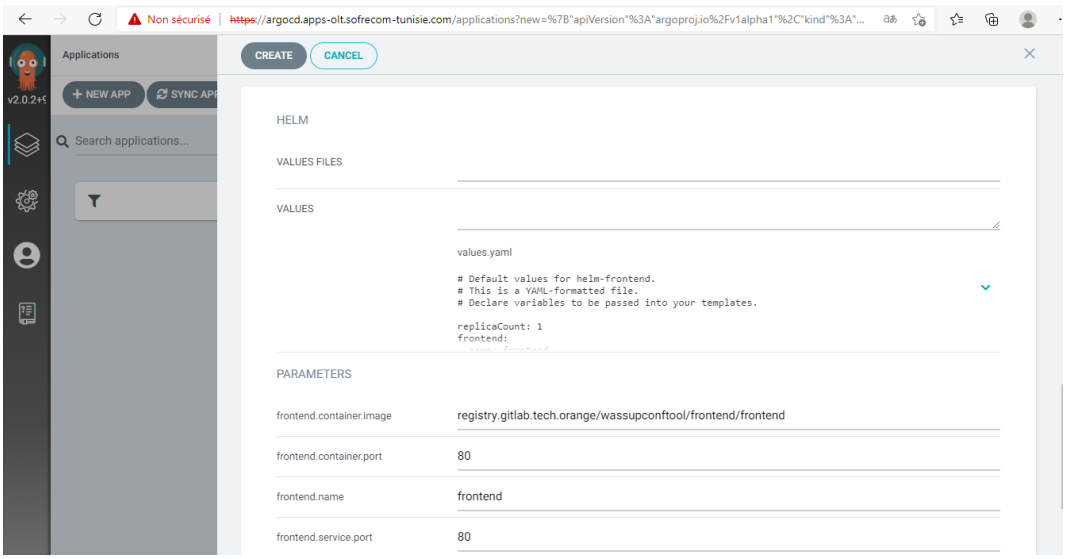


Figure 5.8: Helm Chart section

This section allows us to select the values.yaml file that we want to use, and to override the values in the VALUES part. In our case, we did not need to configure this part.

After the creation of our application, it appeared as an orange tile since we didn't indicate the automatic synchronization at the beginning. When we enabled the automatic sync, the tile turned green as shown in the Figure 5.9 below.

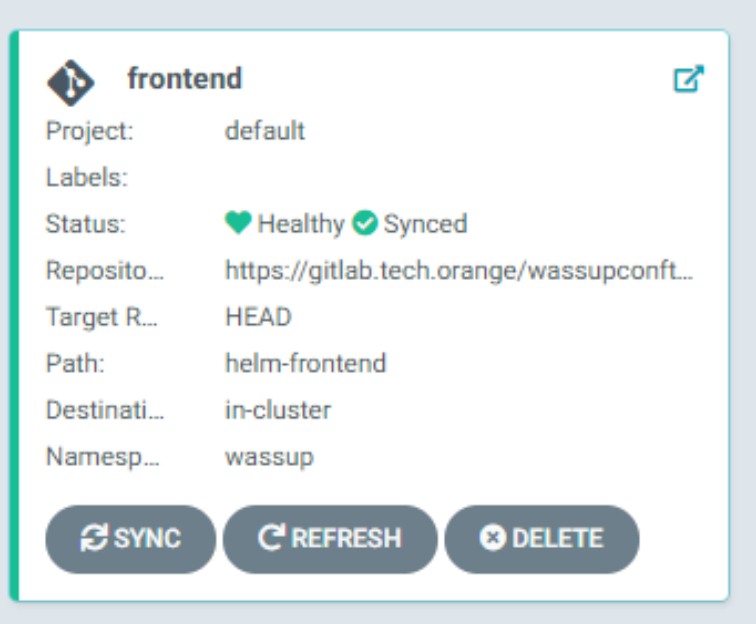


Figure 5.9: The application deployed

To visualize all the Kubernetes objects that we have deployed we clicked on the tile of our application. The Figure 5.10 below shows the set of Kubernetes objects created.

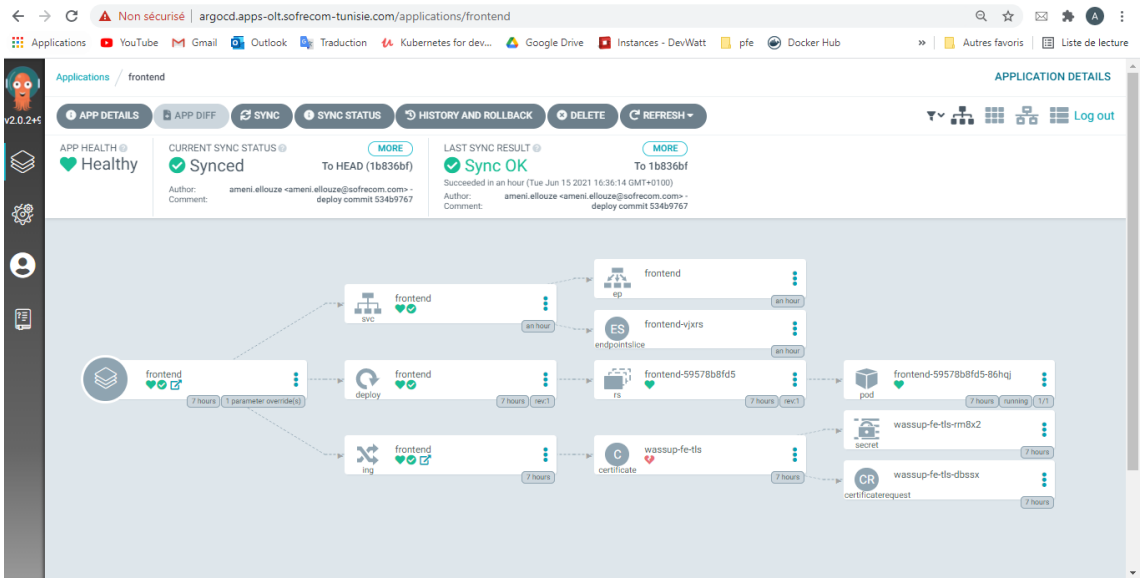


Figure 5.10: Kubernetes objects within ArgoCD's application

This allowed us to see the dependencies between objects, for example: Deployment > Replica Set > Pod, as well as the health of the objects, their revision, the time since their creation.

By clicking on an object, we could see all its events and its YAML configuration as shown in the Figure 5.11 below.

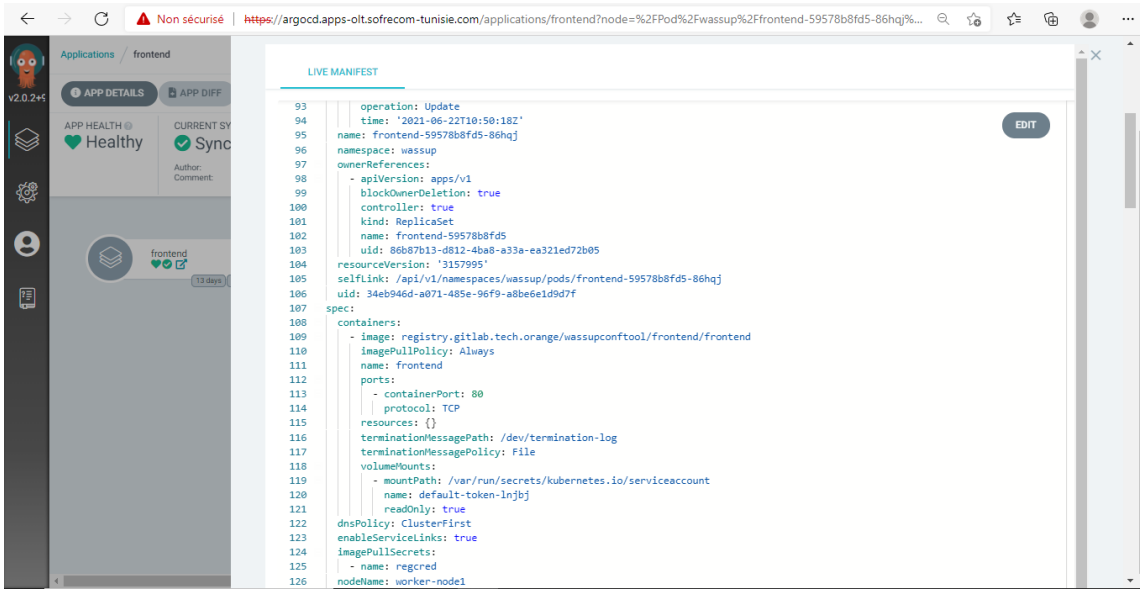


Figure 5.11: Yaml configuration

One main container is running inside each pod, and each pod is deployed on a different node. The image pulled from our registry.gitlab.tech.orange.fr private container registry.

This figure below shows the private container registry used to store our container.

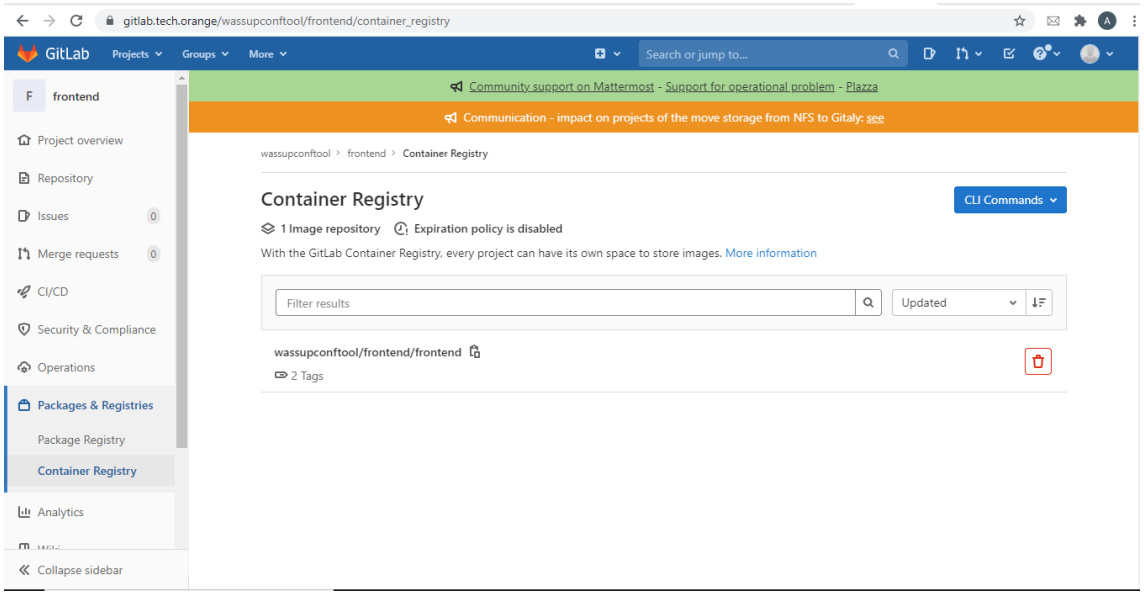


Figure 5.12: Orange Gitlab container registry

After successfully deploying the frontend of the application, we moved to exposing

our services to be able to access them from public external addresses.
The frontend of the Wassup application is shown in the Figure below.

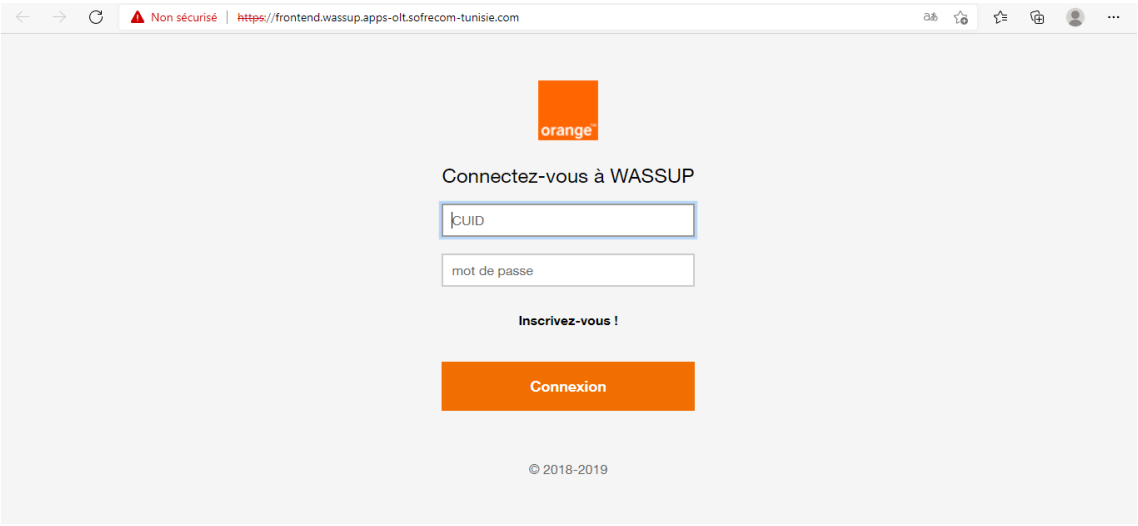


Figure 5.13: The frontend of the application

Our application is deployed in high availability with our Deployment/Service pair and is exposed via a domain name with Ingress. But something is missing: securing our application with TLS, which allows users to securely transmit sensitive data when using the HTTPS protocol.
So we deployed Cert Manager, which is a native certificate manager for Kubernetes. It allows the generation of certificates from different sources like Let's Encrypt, HashiCorp Vault, or a self-signed one. In our case, due to lack of time, we used Let's Encrypt to test, but a Sofrecom certificate should be generated in the next deployment of OLT applications.
We deployed cert-manager using the yamls manifest files and in order not to allow anyone to request a certificate for any CommonName, cert-manager uses the ACME protocol to perform a challenge to prove our identity, so we configured the Let's Encrypt yamls file with our own email as shown in the Figure below

```
[root@master-node ~]# cat letsencrypt.yaml
apiVersion: cert-manager.io/v1alpha2
kind: ClusterIssuer
metadata:
 name: letsencrypt
 namespace: cert-manager
spec:
 acme:
   server: https://acme-v02.api.letsencrypt.org/directory
   email: ameni.ellouze@sofrecom.com
   privateKeySecretRef:
     name: letsencrypt
   solvers:
   - http01:
       ingress:
         class: nginx
[root@master-node ~]#
```

Figure 5.14: Let's Encrypt YAMLS file

We can see the running pods with our kubernetes dashboard as shown in the figure below

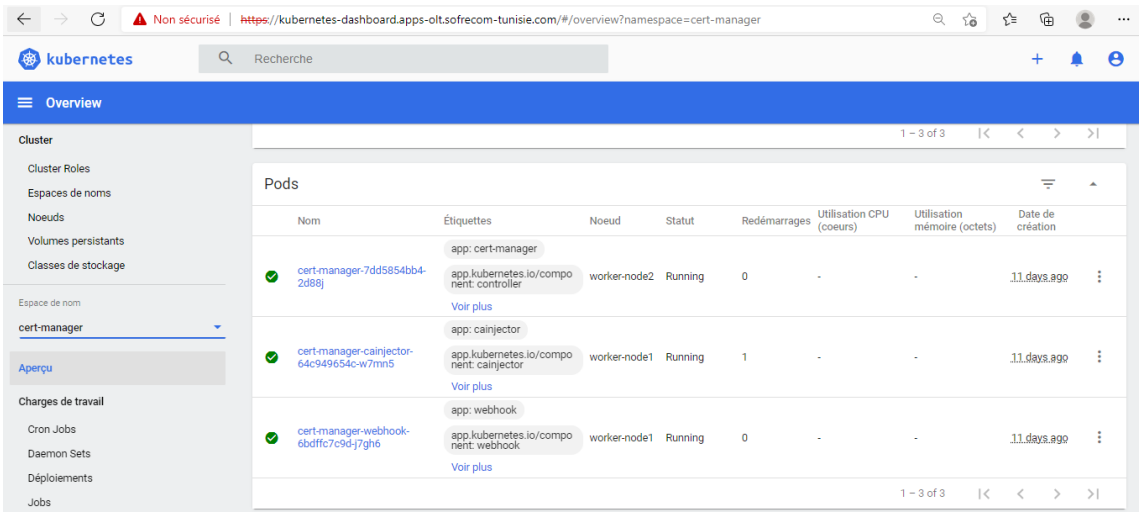


Figure 5.15: Cert-manager Pods

Note that we have used the same process with ArgoCD, Kubernetes dashboard and later with Prometheus, Grafana and AlertManager.

5.3.3 Where GitLab meets ArgoCD

The figure below shows where GitLab CI meets ArgoCD.
Note that the strike-through text represents what we have done in CI parts.

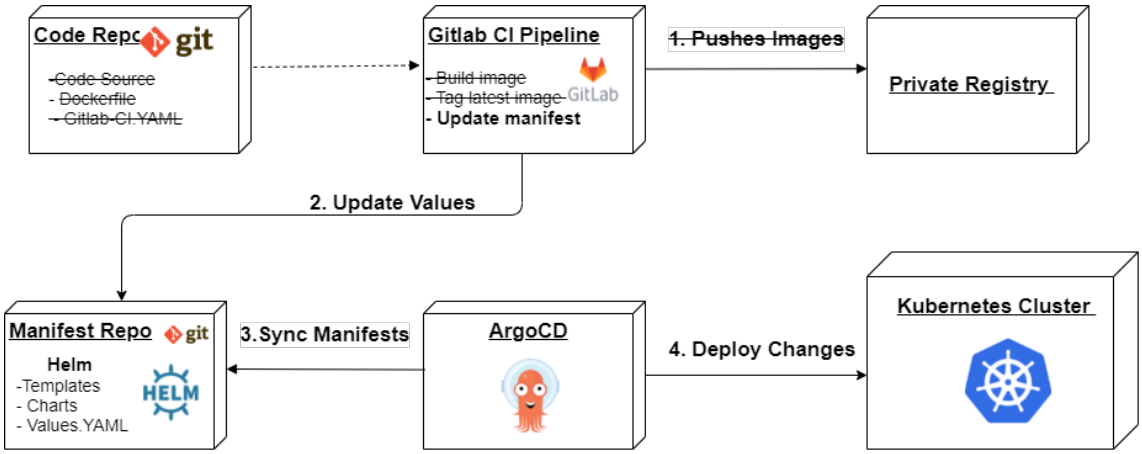


Figure 5.16: GitLab CI meets ArgoCD

ArgoCD does not wait until the new image is built, it just pulls the image again to deploy it if we make changes in the k8s Yamls file.
When the Git repository changes, as a result of a commit, the live state should converge to the desired one. Then, the deployment process is separated from the CI process, since it springs from a change in the Git repository and not necessarily from a natural consequence of CI steps.
So the solution to make ArgoCD detect the new image built after finishing the CI process with gitlab is to add an « Update Manifest » Stage.
We come back to the GitLab CI pipeline again to add another job, named update-manifest

in deploy stage.
This clones and commit new changes to Helm manifest repository. So ArgoCD will detect the changes in the Helm manifest repository and deploy the app again.
Here's how it looks the final CI pipeline of our frontend application in the figure below.

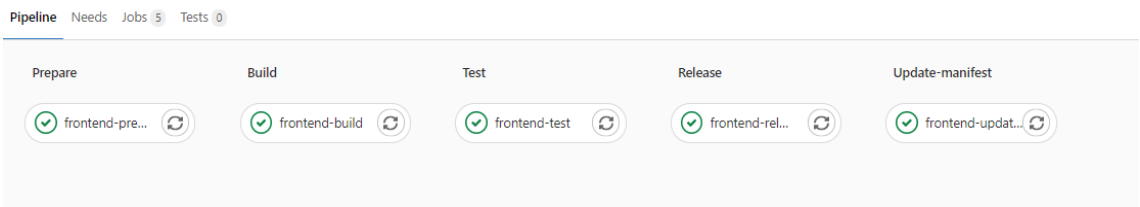


Figure 5.17: CI Pipeline

5.4 Conclusion

To summarise, this chapter presented the deployment process and the tools enabling it namely Kubernetes, Helm, and ArgoCD.
We will now pass to the next step which will define a monitoring platform and enforce the high availability of the k8s cluster.

Chapter 6

Monitoring

Introduction

In this step, we will enforce this high availability with a Prometheus monitoring rich ecosystem. Here are the tools that compose the Prometheus ecosystem and help it perform better:
Prometheus sends notifications to the Alertmanager via custom rules described in configuration files. We export them to a variety of platforms, including Slack and Mattermost. Visualization of data directly using Grafana so we can quickly filter and obtain a clear picture of what's going on with our various goals.

6.1 Deployment Diagram

To better explain this scenario, we have constructed the deployment diagram of the monitoring solution.

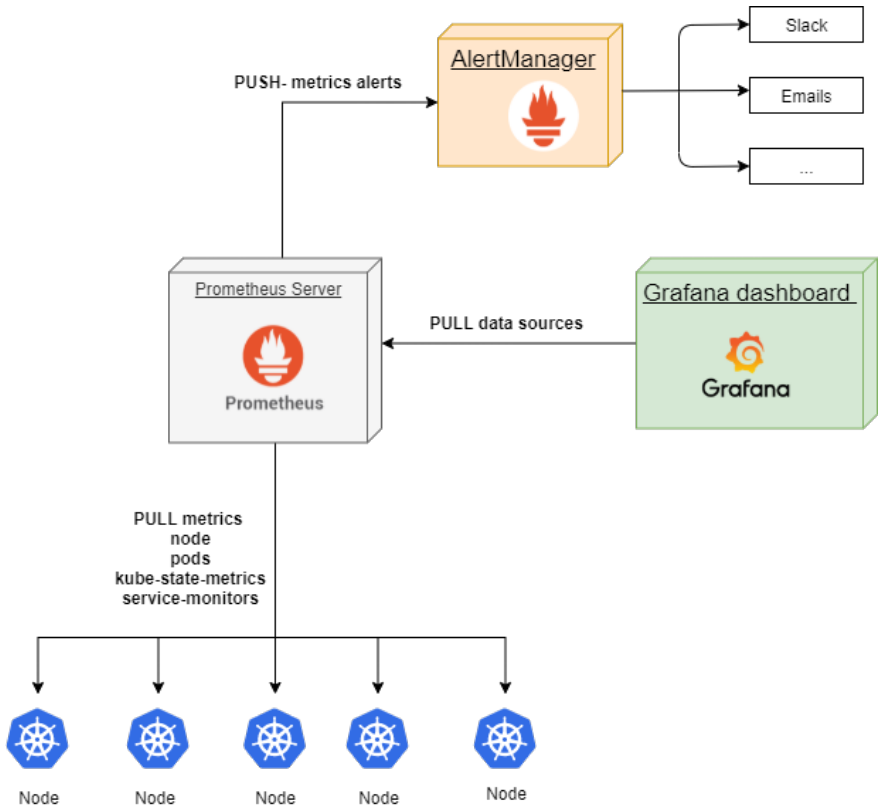


Figure 6.1: Monitoring deployment diagram

- **Prometheus-operator:** orchestrates the monitoring pipeline's different components.
- **Prometheus:** gathers measurements, stores them in a time series database, and provides queries.
- **Alertmanager:** manages alarms issued by client programs like the Prometheus server.

- **Node-exporter:** is installed on each node to gather machine hardware and operating system metrics.
- **Kube-state-metrics:** is a service that communicates with the Kubernetes API server to obtain information on all API objects such as deployments, pods, daemonsets, and so on. Essentially, it offers kubernetes API object metrics that are not available via native Kubernetes monitoring components.
- **Grafana:** is a tool for monitoring and visualizing metrics.
- **Service monitors:** collects internal Kubernetes components.

6.2 Realization

6.2.1 Collecting data with Prometheus

Before deploying Prometheus, we had to make some researches because this collection of tools requires a special treatment to work perfectly and give the bests results without bugging.

Architecture of Prometheus

The Figure below represents the architecture of the Prometheus [50]

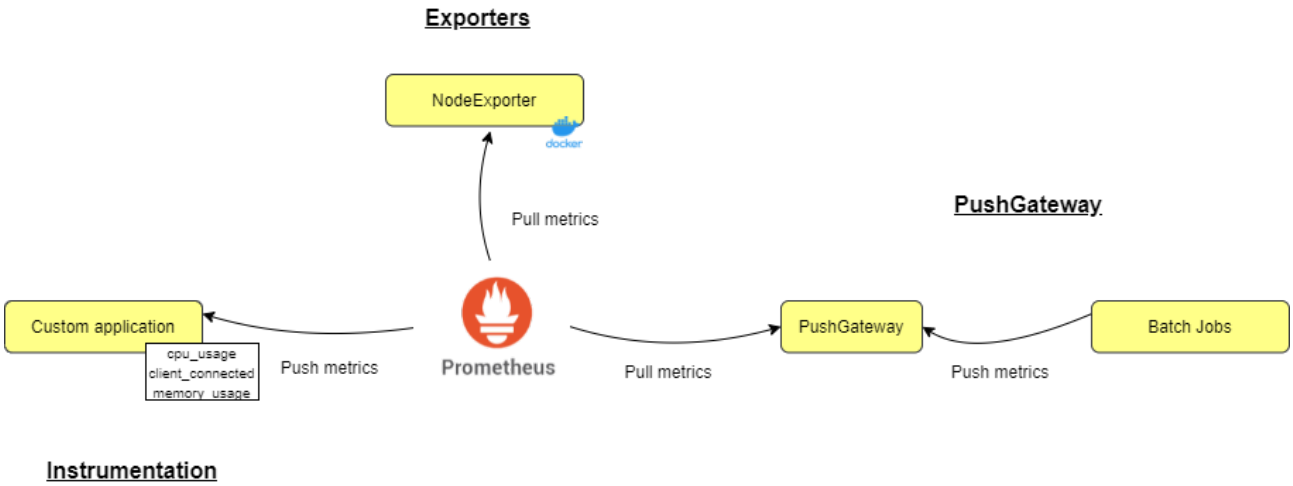


Figure 6.2: The architecture of Prometheus

Prometheus, as indicated previously in the deployment diagram, is made up of a wide range of distinct components.
First and foremost, we need to be able to retrieve metrics from our systems. We can accomplish it in a variety of ways:

- By 'instrumenting' our application, we imply that we will offer Prometheus-compatible metrics on a specific URL. Prometheus will classify it as a target.

- By using prebuilt 'Exporters': Prometheus includes a large library of exporters for current technologies. In our situation, we utilize prebuilt exporters to monitor Linux machines (node exporter).
- Using the 'Pushgateway': sometimes when we have apps or tasks that do not offer metrics directly. Certain apps are either not intended for it (for example, batch tasks), or we have chosen not to offer those metrics directly through our app.

So, Prometheus is a pull-based monitoring system if we overlook the few circumstances where we could utilize Pushgateway.

Prometheus deployment

After creating a new namespace "monitoring", we deployed Prometheus with sample yamls files, like we did with the dashboard deployment, (Deployments, Service, Ingress, Configmap and ClusterRole). So we can add any use case to retrieve metrics from any other object to the our cluster role.
We also deployed Kube state metrics service that will provide many metrics which are not available by default, and Node Exporter that will provide all the Linux system-level metrics of all Kubernetes nodes.

And the Prometheus dashboard is presented in the Figure below.

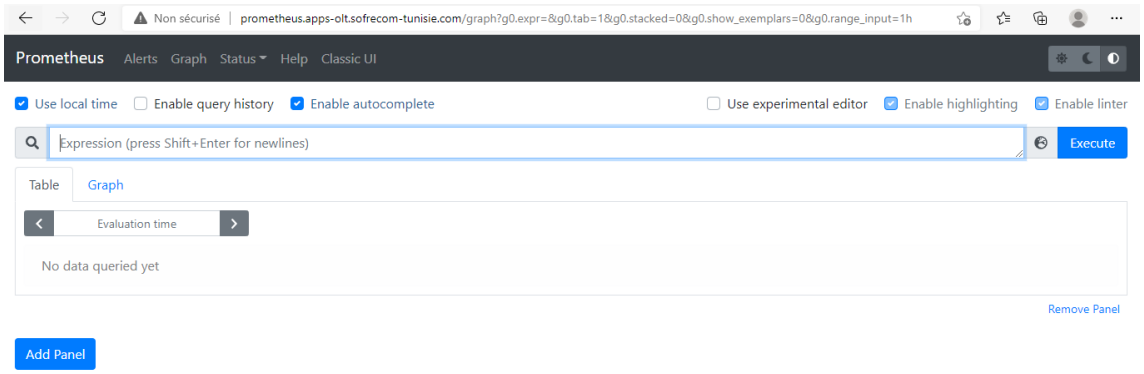


Figure 6.3: Prometheus dashboard

We can quickly visualize time series metrics using this Prometheus Browser by showing them on a graph or in a tabular format, depending on our choices.
By putting the metric name into the Expression area in the Browser, we can select which specific measure we want to display. When we click the Execute button, data for this metric is displayed in both a table (on the Console Tab), presented in the Figure 6.4, and a graph (on the Graph Tab), presented in the Figure 6.5, with the ability to move between the two with a single click.

In the following example, we want to extract data on the prometheus target interval length seconds metric. The time between target scrapes is measured by this metric. In other words, the period elapsed between data collection from a Prometheus target.

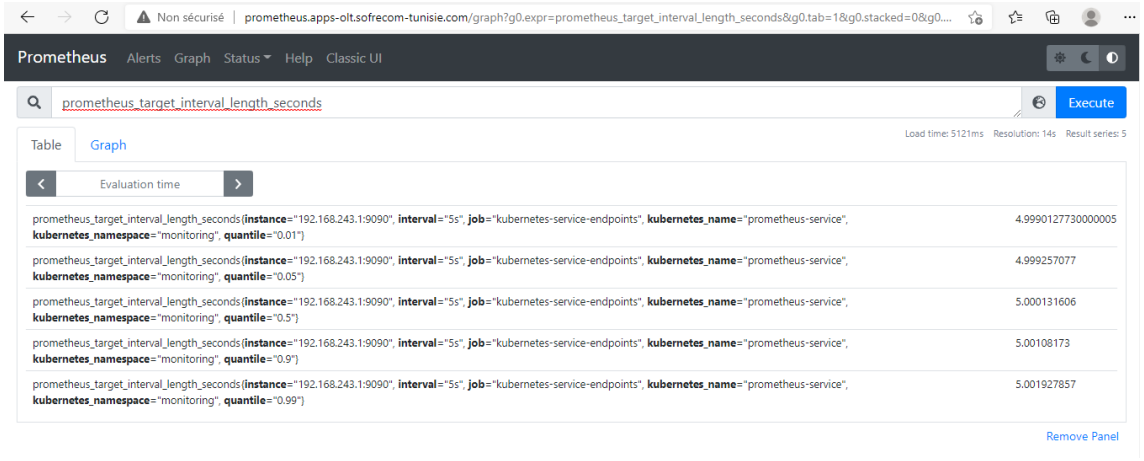


Figure 6.4: Prometheus Targets displayed in a Table



Figure 6.5: Prometheus Targets displayed in a Graph

From this dashboard, we can access the targets to control the monitor services, as shown in the figure below. There may be a lot of targets since Prometheus has been set to look at all service endpoints.

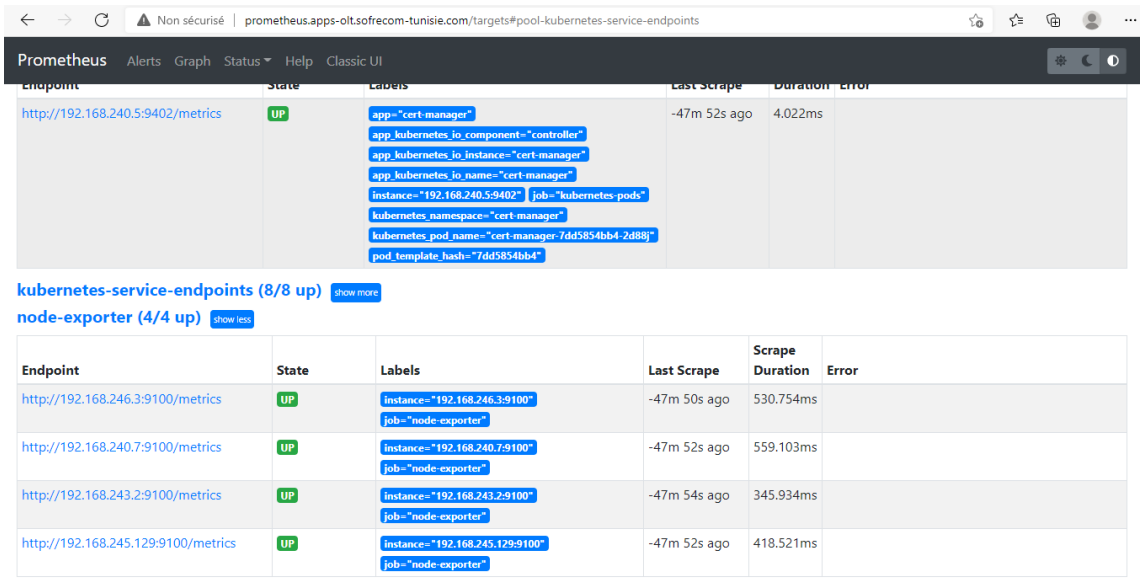


Figure 6.6: Prometheus Targets

6.2.2 Sending notifications to the AlertManager

AlertManager is one binary that processes Prometheus server alarms and notifies end-users through email, Slack, or other methods. Monitoring helps to detect prospective problems or to inform our system about present problems and details the situation. Alerting helps alert teams when the problem occurs and enables notifications to detect problems.
We deployed AlertManager using the manifest files like we did with prometheus.

We can see the details from the kubernetes dashboard like shown below.

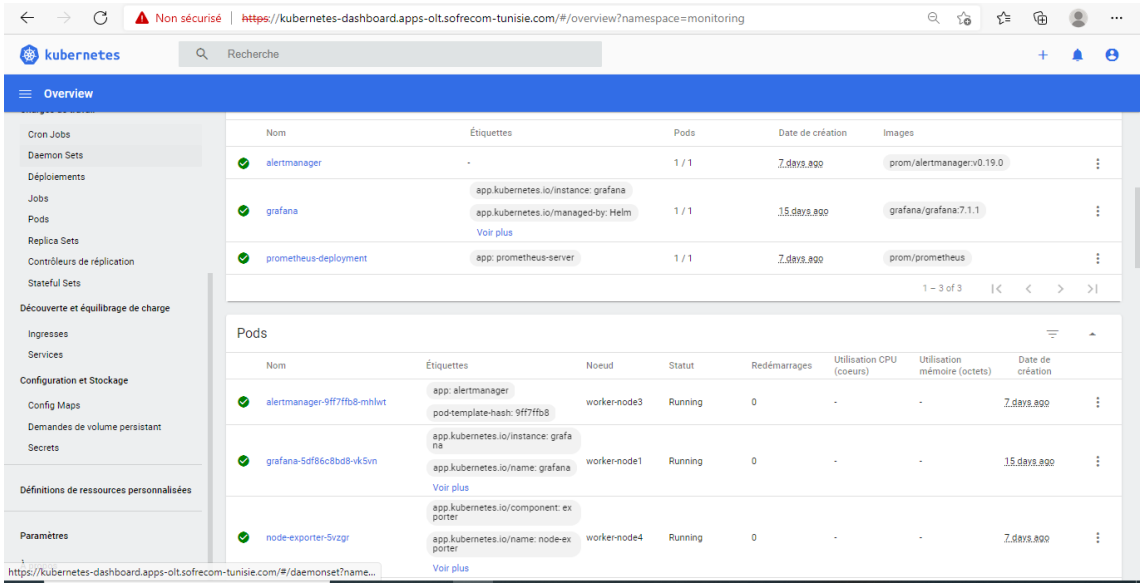


Figure 6.7: Prometheus deployed

The Figure below represents the AlertManager's dashboard.

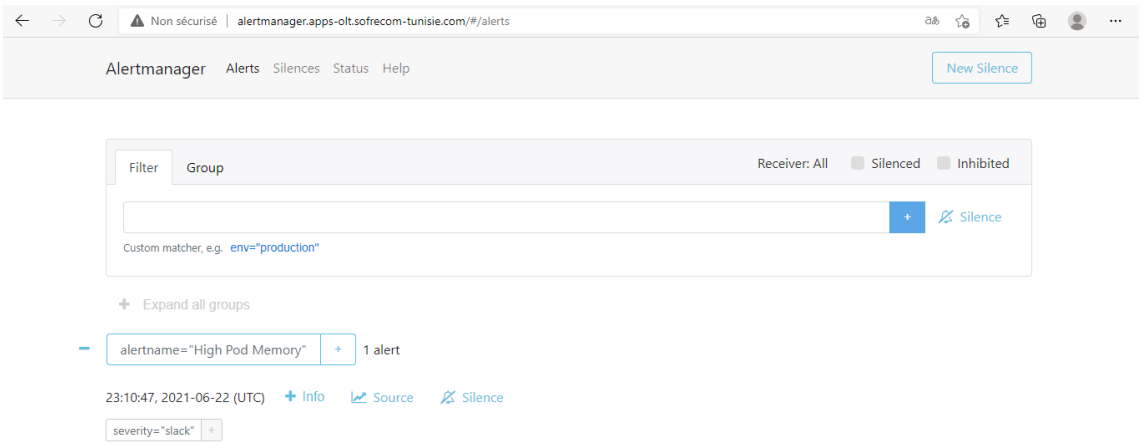


Figure 6.8: AlertManager dashboard

Creating a test alert

To add a test alert with Slack as shown in the Figure above, we created a config.yaml file which describes the data of the receivers such as email, api-url of Slack.
This alert allows us to identify if there will be High Pod Memory Usage.
We can see the alert rule in the prometheus web interface, as shown in the figure below, by going to the Alerts section.

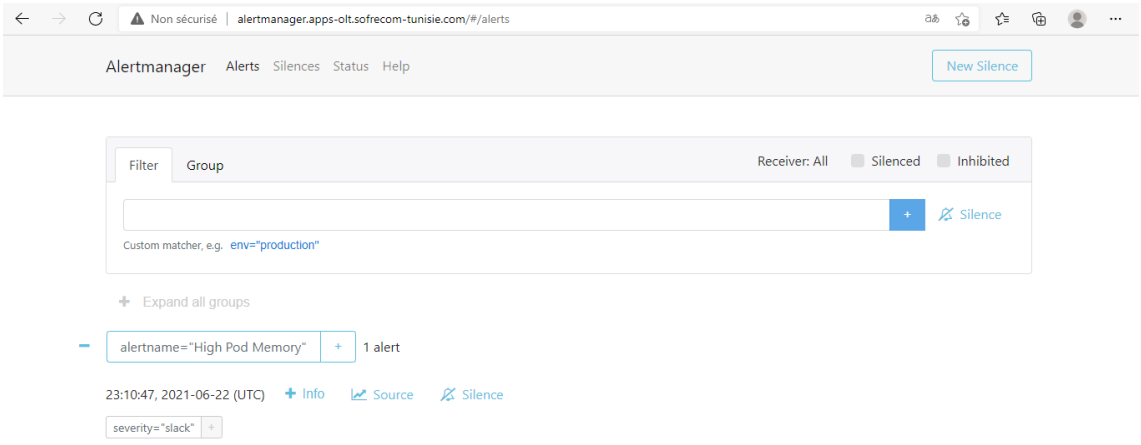


Figure 6.9: AlertManager dashboard

We had a ”Firing” alert because we had high usage of the container’s memory. And we received the alerting message in Slack like as shown in the Figure 6.10 below.

In order to test all the functionalities within AlertManager, we choose to silence the alert, to prevent any more alerts from being sent out (Figure 6.11).
We were able to customize the period of the silence, add a creator and a description for additional metadata, and extend or limit the group of alerts to which this silence applies.

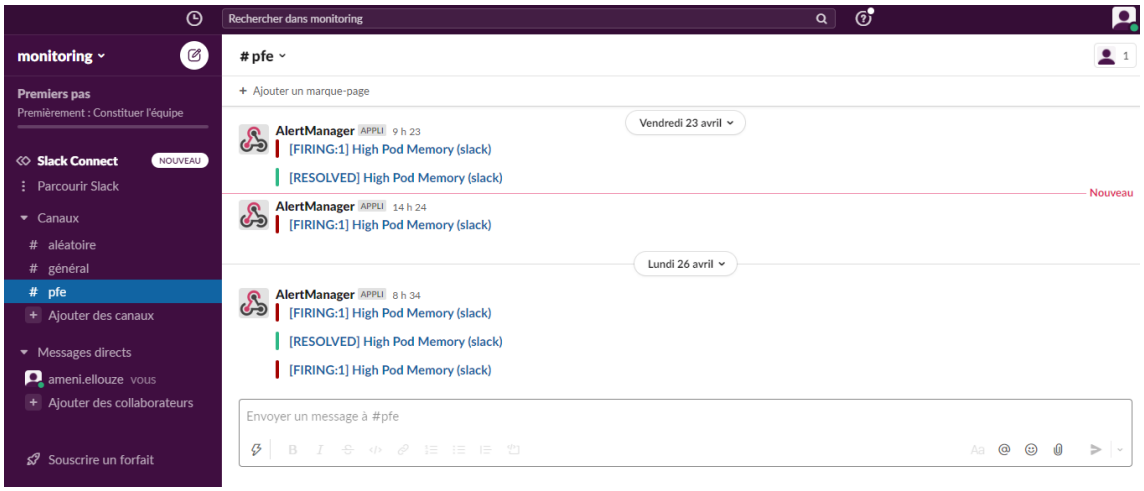


Figure 6.10: Slack alerting message

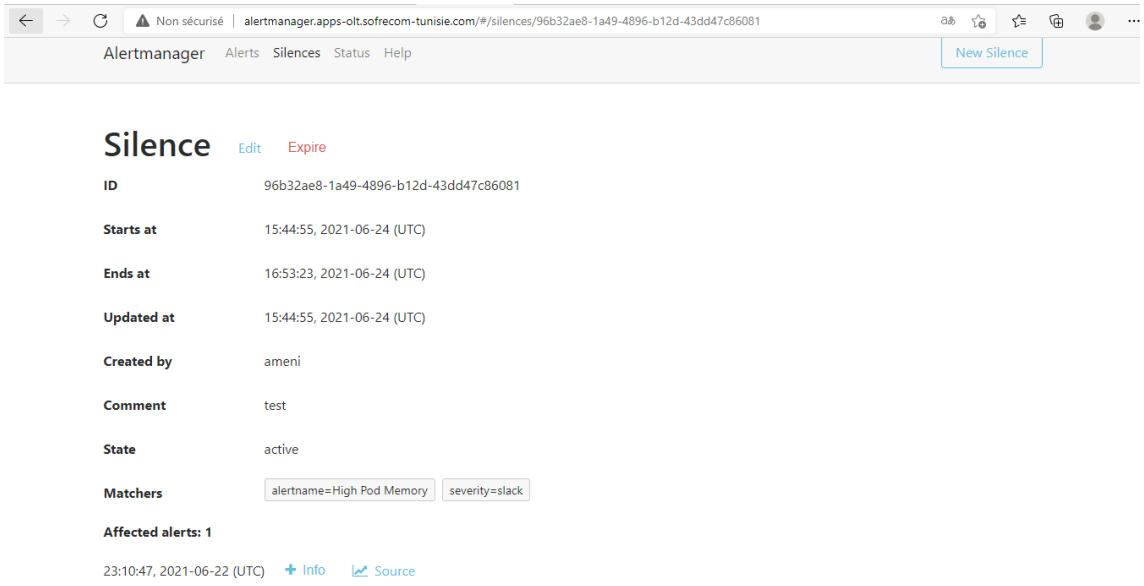


Figure 6.11: AlertManager Silence

6.2.3 Visualizing data with Grafana

We augment the Kubernetes resource monitoring process with Prometheus by installing Grafana and utilizing the Prometheus data source to build information-rich dashboards in an easy-to-use detailed format.
We deployed Grafana using ArgoCD, as we did with Prometheus. We connected Argo with "Stable" Helm and created the Grafana chart as represented by the Figure below.

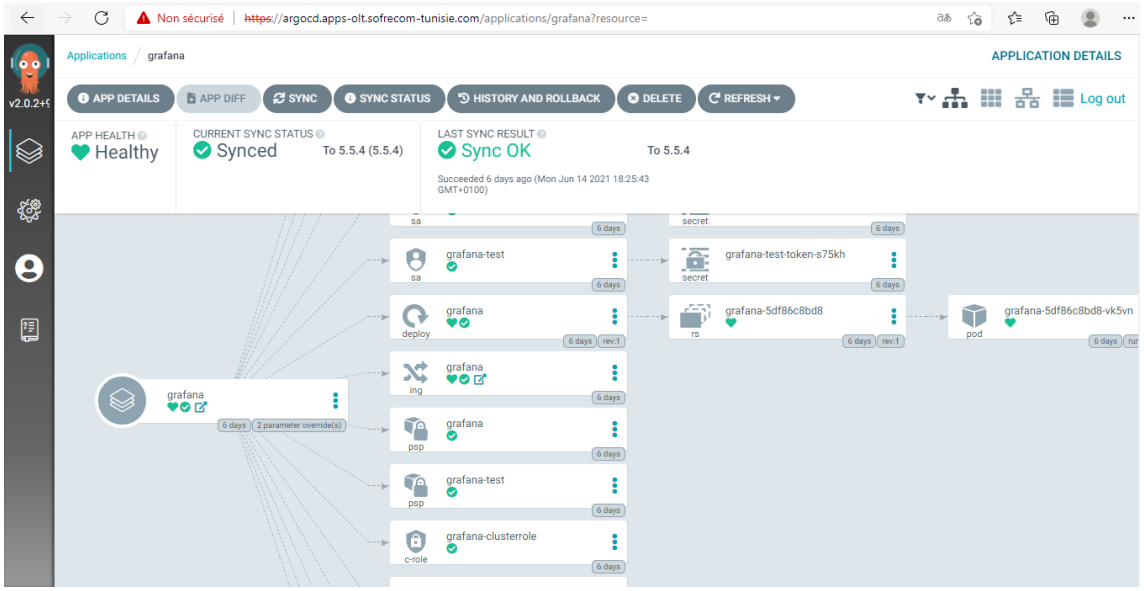


Figure 6.12: Grafana application deployed

After configuring the settings for Grafana, we were able to access our dashboard through the ingress URL host that we have configured as shown below:

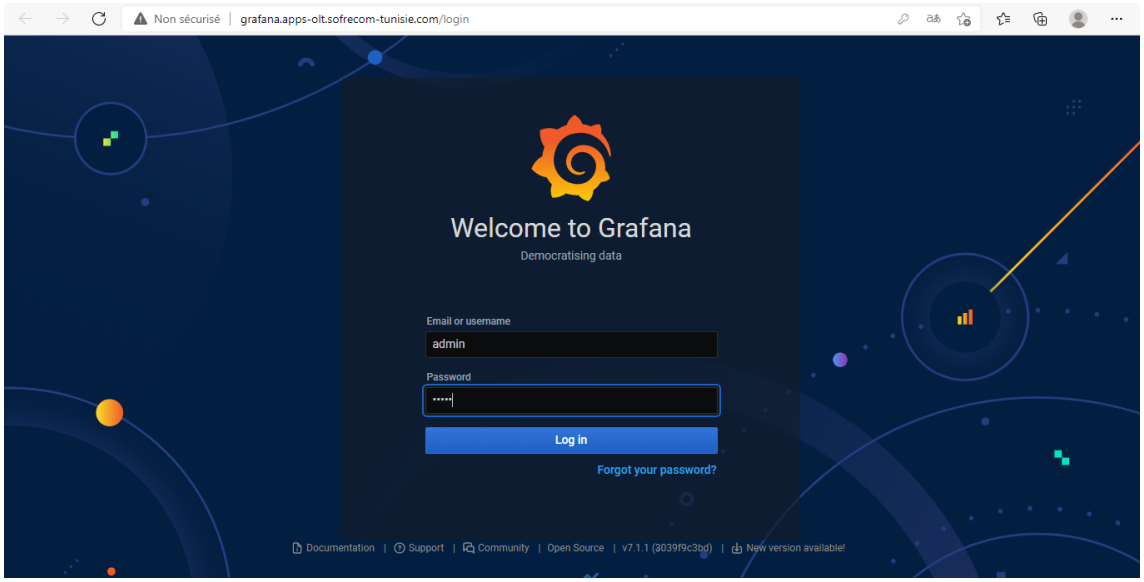


Figure 6.13: Grafana logging dashboard

The next step was to build a new data source in Grafana that connected to our Prometheus installation, as seen in the figure below.
To accomplish this, we had to go to Configuration > Data Sources and select the blue Add data source option.
Grafana supports a variety of time series data sources, however, we choose the most popular, Prometheus. We provided the URL of our Prometheus server, but we kept the other fields blank because they aren't required.

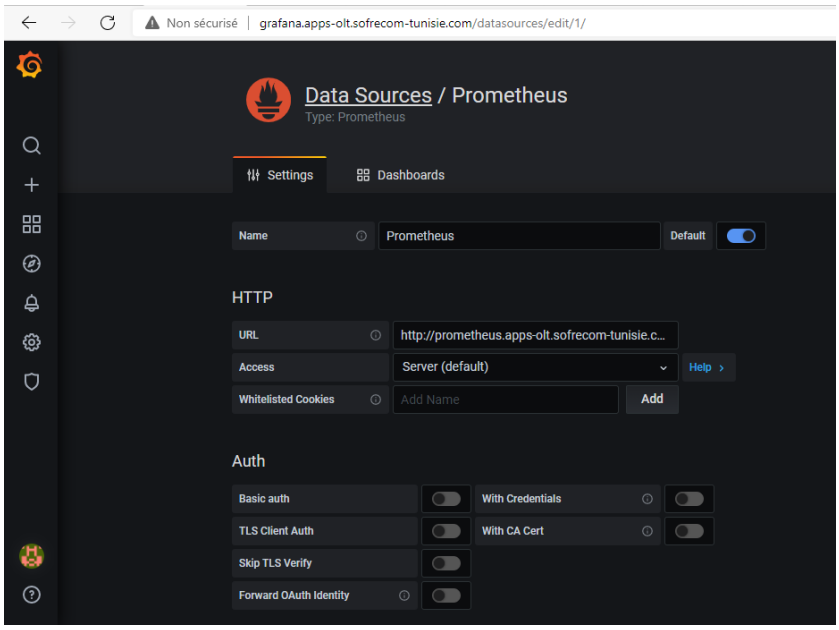


Figure 6.14: Grafana data source

We used one of the dashboards produced by the Grafana-community which provided us with a beautiful overview of the metrics collected from the node exporter like is shown in the figure below.



Figure 6.15: Grafana dashboard

6.3 Conclusion

In this step, we implemented a monitoring platform (Prometheus, AlertManager and Grafana), for our kubernetes cluster and the OLT applications to review all its logs, make sure that it's running perfectly and enforce its high availability.

General Conclusion

General Conclusion

GitOps has clear and measurable benefits: lower production cost, more efficiency, and enhanced productivity. Applying its philosophy and methodologies drives the industry forward. We had the chance to install the GitOps approach in the host company Sofrecom Tunisia by testing new technologies alongside their proper solutions. Our graduation project is about creating a production-ready kubernetes cluster with a continuous deployment process through GitOps techniques and implementing a whole monitoring platform.

During the development of this project, our work had two major steps. First, we elaborated the High-Level design and architecture of the platform so we could set up an infrastructure of suitable hardware and software components. We also set up a collaboration workflow based Agile, Kanban, and DevOps. Then, we started designing and implementing the Continuous Delivery and Deployment of the different components of a pilot application.

To accomplish this work, we presented in the report the general context of the project, the actual solution, and our objectives and contributions. After that, we presented the state of the art which explained the main concepts for project realization. Then, we detailed the analysis and conception and we presented the realization of each step of the project.

The many new concepts to learn were a tough challenge: we had to initiate each one of them with some preliminary studies by exploring different official and unofficial documentations to extract the best of each concept and tool and know how to perfectly implement them.

Even that our project was accomplished successfully, there's still some work to do, we have to set up the tuning and system security part and update the compute limits to avoid VMs crash and finally secure the ssh access.

Netography

[1] Agile Aliance. *Kanban*. URL: https://www.agilealliance.org/glossary/kanban/#q=~(infinite~false~filters~(postType~(~'page~'post~'aa_book~'aa_event_session~'aa_experience_report~'aa_glossary~'aa_research_paper~'aa_video)~tags~(~'kanban))~searchTerm~'~sort~false~sortDirection~'asc~page~1).

[2] Actia solutions. *Kanban Methodology*. URL: https://aktiasolutions.com/kanban-methodology-kanban-method/.

[3] *Kanban events*. URL: https://aktiasolutions.com/wp-content/uploads/2019/07/Kanban-Cadences-Kanban-Feedback-Loops-Kanban-Method-Kanban-Methodology-Metodo-Kanban-Metodologia-Kanban-AKTIA-Solutions-768x599.png.

[4] Ben Straub Scott Chacon. *Pro Git*. 2021-06-14. URL: https://git-scm.com/book/en/v2.

[5] the agile admin. *Definition of DevOps*. URL: https://theagileadmin.com/what-is-devops/.

[6] Emily Mell Meredith Courtemanche and Alexander S. Gillis. *What is DevOps, the ultimate guide*. 2020. URL: https://searchitoperations.techtarget.com/definition/DevOps.

[7] *Continous Integration*. URL: https://iclerisy.com/wp-content/uploads/2019/08/Continuous-Integration.png.

[8] *Continuous Deployment and Continuous Delivery*. URL: https://newrelic.com/sites/default/files/wp_blog_inline_files/ci_cd.jpg.

[9] *What is Infrastructure as Code*. URL: https://docs.microsoft.com/en-us/azure/devops/learn/what-is-infrastructure-as-code.

[10] *DevOps Process*. URL: https://miro.medium.com/max/1050/0*n57zykBMdOdmUg7C.png.

[11] *Guide To GitOps*. URL: https://www.weave.works/technologies/gitops/.

[12] *Push Based Pipeline*. URL: https://www.gitops.tech/.

[13] *Push-based pipeline*. URL: https://www.gitops.tech/images/push.png.

[14] *Pull-based pipelin*. URL: https://www.gitops.tech/images/push.png.

[15] *The History of Virtualization and Containerization*. URL: https://kubernetestutorials.com/kubernetes-beginners-tutorials/the-history-of-virtualization-and-containerization/.

[16] *VMs vs Container*. URL: https://blog.netapp.com/wp-content/uploads/2016/03/Screen-Shot-2018-03-20-at-9.24.09-AM.png.

[17] Medium. *The What, Why, and How of a Microservices Architecture*. URL: https://medium.com/hashmapinc/the-what-why-and-how-of-a-microservices-architecture-4179579423a9.

[18] *Microservices vs Monolithic architecture*. URL: https://i.pinimg.com/originals/96/0f/a5/960fa5a8e6f117196c6e6c7a7ab7c70f.png.

[19] IT monitoring. *The definitive guide to enterprise IT monitoring*. URL: https://searchitoperations.techtarget.com/definition/IT-monitoring.

[20] *Docker's logo*. URL: https://sys-admin.fr/wp-content/uploads/2020/06/docker-logo.png.

[21] Docker docs. *Docker overview*. URL: https://docs.docker.com/get-started/overview/.

[22] *Gitlab's logo*. URL: https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcSjFzv4zGZjD2X3CQARzu4c5dVkI4_X3kTf8pGp8F_r54ewFy77v-Aa98XG9SOSGtoM3DQ&usqp=CAU.

[23] *Kubernetes's logo*. URL: https://ml32ltopxlgp.i.optimole.com/j9-pdEk-NyyhK8p-/w:auto/h:auto/q:75/https://www.blackcreeper.com/wp-content/uploads/2020/04/kubernetes-logo-big.png.

[24] *CI/CD Gitlab's logo*. URL: https://gieglas.com/wp-content/uploads/2019/08/gitlab-cicd.png.

[25] *GitLab CI/CD*. URL: https://docs.gitlab.com/ee/ci/.

[26] *ArgoCD's logo*. URL: https://particule.io/images/argo/argo-logo.png.

[27] *Argo CD - Declarative GitOps CD for Kubernetes*. URL: https://argoproj.github.io/argo-cd/.

[28] *Lens' logo*. URL: https://github.com/lensapp.png.

[29] *Lens, The Kubernetes IDE*. URL: https://k8slens.dev/.

[30] *Grafana logo*. URL: https://reseau.probesys.com/sites/default/files/2020-05/grafana.png.

[31] Grafana. *Dashboard anything. Observe everything*. URL: https://grafana.com/grafana/.

[32] *Prometheus logo*. URL: https://aperogeek.fr/wp-content/uploads/2018/05/logo_prometheus_padding-300x277.png.

[33] *AlertManager logo*. URL: https://prometheus.io/assets/favicons/android-chrome-192x192.png.

[34] Prometheus. *Alertmanager*. URL: https://prometheus.io/docs/alerting/latest/alertmanager/.

[35] *Helm logo*. URL: https://blog.wescale.fr/content/images/2018/05/Helm-Logo_Plan-de-travail-1.png.

[36] Helm. *The package manager for Kubernetes*. URL: https://helm.sh/.

[37] *Nginx logo*. URL: https://dwglogo.com/wp-content/uploads/2017/09/3630px-Nginx_logo.png.

[38] Red Hat. *NGINX Ingress Operator*. URL: https://catalog.redhat.com/software/operators/detail/5e9874913f398525a0ceb00d.

[39] *Ansible logo*. URL: https://upload.wikimedia.org/wikipedia/commons/thumb/2/24/Ansible_logo.svg/1200px-Ansible_logo.svg.png.

[40] Red Hat. *Drive automation across open hybrid cloud deployments*. URL: https://www.ansible.com/.

[41] *Mariadb logo*. URL: https://mariadb.com/wp-content/uploads/2019/11/mariadb-logo-vert_blue-transparent.png.

[42] MariaDB. URL: https://mariadb.org/.

[43] OpenEBS. URL: https://openebs.io/.

[44] *VS Code logo*. URL: https://upload.wikimedia.org/wikipedia/commons/thumb/9/9a/Visual_Studio_Code_1.35_icon.svg/1200px-Visual_Studio_Code_1.35_icon.svg.png.

[45] Visual Studio Code. URL: https://code.visualstudio.com/docs/editor/whyvscode.

[46] *Bash Scripting logo*. URL: https://upload.wikimedia.org/wikipedia/commons/thumb/4/4b/Bash_Logo_Colored.svg/1200px-Bash_Logo_Colored.svg.png.

[47] *Draw.io logo*. URL: https://i.pcmag.com/imagery/reviews/07vFb2AaL0wT9gxtJbAqUht-5..1569473381.jpg.

[48] *icrosoftPowerPoint logo*. URL: https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQYdJrYzCJ64hmqG9Z7UR4An4Ps2spTNmBjU04qfXMJfbUfsUR8pcApJtK_kKsyOVsvkYk&usqp=CAU.

[49] Andrew Block. *Learn Helm. Packt*. 2020. URL: https://subscription.packtpub.com/book/cloud_and_networking/9781839214295.

[50] Devconnected. URL: https://devconnected.com/the-definitive-guide-to-prometheus-in-2019/.

Appendices

Appendix A

Kubernetes

A.1 Introduction

Kubernetes (also known as k8s or "kube") is an open-source container orchestration platform that automates many of the manual processes involved in deploying, managing, and scaling containerized applications.

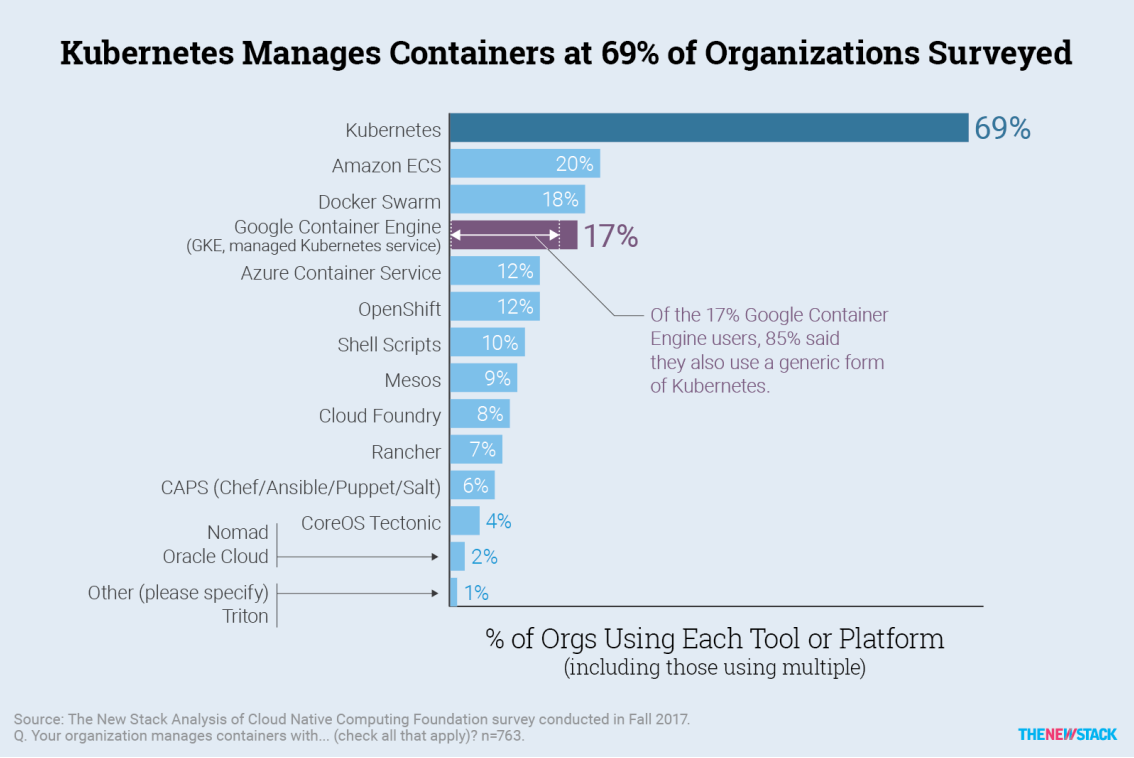


Figure A.1: Kubernetes adoption

Kubernetes launched in 2014 as an open-source version of Google's internal orchestrator Borg. Google first launched Borg in 2003-2004. It was a large-scale internal cluster management system that handled a massive number of jobs from a massive number of applications across several clusters, each with tens of thousands of workstations.
Kubernetes was deployed by several enterprises beginning in 2017-2018. One of the rea-

sons for Kubernetes' rapid popularity is its open source design, as well as the enormous amount of documentation and help supplied by its community. There are several Kubernetes distributions available. There is a vanilla version, which is the most basic, as well as several others, such as RedHat OpenShift, Tectonic, Rancher, and others.

A.2 Architecture

The k8s architecture is shown in Figure A.2. A cluster is a collection of resources managed by a single instance of k8s. Each Kubernetes cluster is made up of either a master node and one or more worker nodes.

- **Master node:** It must exist since it is the focal point of control for the whole cluster. It is possible to replicate it for redundancy and load balancing.
  It maintains and controls a group of worker nodes. If the cluster is just made up of one node, it can also function as a worker node.

- **Worker node:** This node is responsible for hosting the pods, which are the components of the application workload. It also has agents that communicate with the master node.

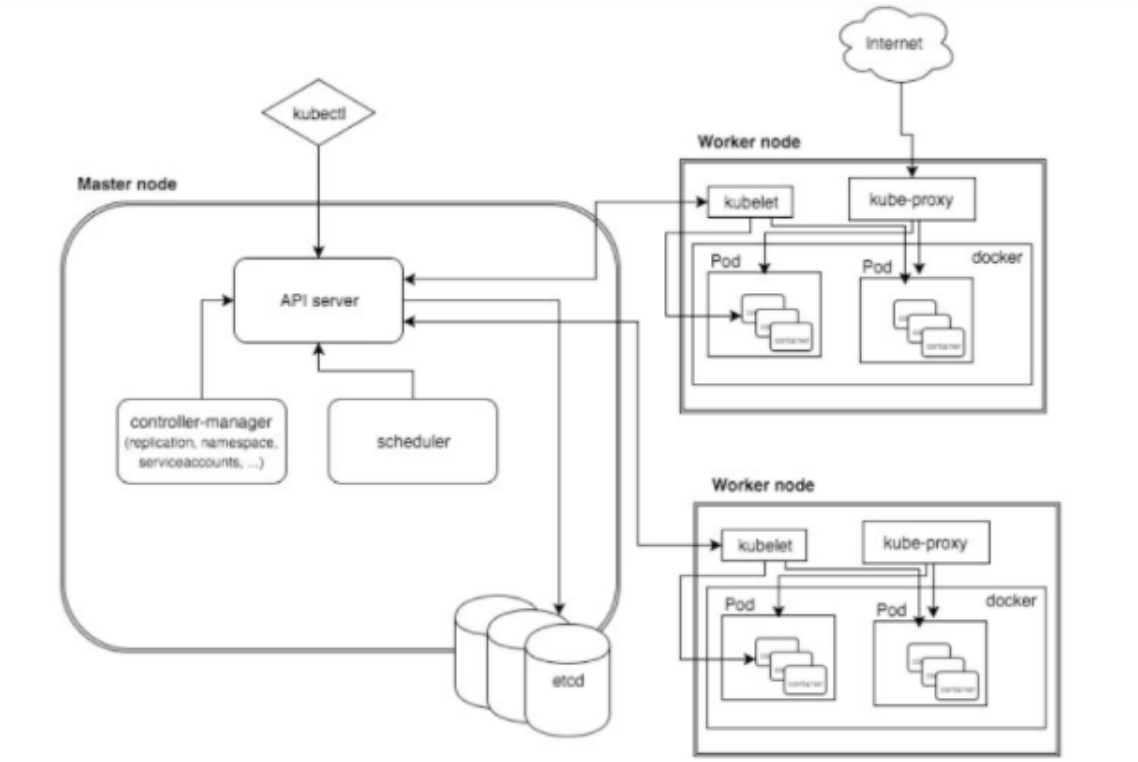


Figure A.2: Kubernetes architecture

Master Node Components

The control plane unit of a Kubernetes cluster, the master node, is in charge of making global decisions, maintaining the state of all objects in the cluster, and responding to events. It runs control loops indefinitely to observe changes.
The master node is the component through which users interact with Kubernetes. Clients submit requests, and it ensures that the cluster's current state is changed to the desired one. The components of the Master node are as follows:

- **API Server:** Kubernetes API server is the central management entity that receives all REST requests for modifications (to pods, services, replication sets/controllers and others), serving as a frontend to the cluster. Also, this is the only component that communicates with the etcd cluster, making sure data is stored in etcd and is in agreement with the service details of the deployed pods.
- **Controller Manager:** runs several distinct controller processes in the background (for example, replication controller controls the number of replicas in a pod, endpoints controller populates endpoint objects like services and pods, and others) to regulate the shared state of the cluster and perform routine tasks. When a change in a service configuration occurs (for example, replacing the image from which the pods are running, or changing parameters in the configuration yaml file), the controller spots the change and starts working towards the new desired state.
- **etcd:** a simple, distributed key value storage which is used to store the Kubernetes cluster data (such as the number of pods, their state, namespace, etc), API objects and service discovery details. It is only accessible from the API server for security reasons. etcd enables notifications to the cluster about configuration changes with the help of watchers. Notifications are API requests on each etcd cluster node to trigger the update of information in the node's storage.
- **Scheduler:** helps schedule the pods (a co-located group of containers inside which our application processes are running) on the various nodes based on resource utilization. It reads the service's operational requirements and schedules it on the best fit node.

Worker Node Components

Worker nodes are simpler than Master node. Each worker node hosts Pods and two daemons:

- **Container Runtime:** Container runtime is in charge of container execution. Kubernetes supports a variety of options, the most prevalent of which being Docker.
- **Kubelet:** the main service on a node, regularly taking in new or modified pod specifications (primarily through the kube-apiserver) and ensuring that pods and their containers are healthy and running in the desired state. This component also reports to the master on the health of the host where it is running.

- **Kube-Proxy:** a proxy service that runs on each worker node to deal with individual host subnetting and expose services to the external world. It performs request forwarding to the correct pods/containers across the various isolated networks in a cluster.

Kubectl

Kubectl command is a line tool that interacts with kube-apiserver and sends commands to the master node. Each command is converted into an API call.

Kubernetes Concepts

To use Kubernetes, we need to understand all the abstractions it utilizes, such as services, pods, volumes, namespaces and installations, to describe the system's state.

- **Pod:** Generally refers to one or more containers that should be controlled as a single application.
  A pod contains apps, storage resources, a unique network ID, and other container running configurations.

- **Service:** Pods are volatile, which means that Kubernetes does not guarantee the maintenance of a specific physical pod (for instance, the replication controller might kill and start a new set of pods). Instead, a service is a logical group of pods, acting as a gateway to enable (client) pods to submit requests to the service without tracking the service's physical pods.

- **Volume:** Similar to a container volume in Docker, except the Kubernetes volume applies to a full pod and is mounted on every container in the pod. Kubernetes assures that all container restarts are maintained. Only after the pot is destroyed will the volume be eliminated.

- **Namespace:** A virtual cluster (one cluster can operate many virtual clusters) designed for various teams and project isolations in situations with numerous users. Resources inside the namespace must be unique and resources in a different namespace cannot be accessed. A resource limit can also be allocated to avoid utilizing a greater portion of the total resources of the physical cluster.

- **Deployment:** Describes the intended state of a yaml file of a pod or replica. The deployment controller then adjusts the environment progressively (for instance, replica creation or deletion) until the current state matches the state of the installation file.

Appendix B

ArgoCD

B.1 Introduction

ArgoCD is a declarative, GitOps continuous delivery tool used in Kubernetes. Declarative and version controlled applications, settings and environments should be established. Automated, auditable and easy to comprehend applications and lifecycle management. ArgoCD embraces the concept GitOps entirely, thus it is used as a unique source of truth together with Git repositories. The user has the versions control, which is an intrinsic Git feature as a collateral consequence. In addition, it is a native Cloud tool because it is built to operate the Kubernetes. Since the initial edition was released in March 2018, the technology is relatively young.
The main goal of this technology is to automate the deployment process of applications.

B.2 How it works?

In general, we may construct an ArgoCD manifest which is a declarative yaml file for deploying an application in the Kubernetes cluster. Inside it, the remote Git repository we wish to connect to must be specified. The repository may be Github or a GitLab repository and so on, as the ArgoCD manifest needs only the https URL to connect. It does not matter which one to use. The k8s manifestations may be described inside the repository in various methods, such as custom application, Helm chart, ksonnet applications, yaml/Json simple files, ksonnet apps or a custom management tool. The destination clusters, where the application is intended to be deployed, are another key element to be given.
On kubernetes the ArgoCD manifest may be deployed. It represents a definition of custom resource and is known as an application. The Git application connected to the repository contains all of the required yaml files to deploy the app within the k8s cluster. ArgoCD checks for any changes in the repository once the application has been deployed. If updates are found, the deployed program will remain automatically in the desired remote state.

For instance, if a user decides to increase the number of replicas of a specific kubernetes Deployment object, he can make it directly on the Git repository, committing and pushing the change in the specific yaml file. ArgoCD will notice inside the repository and converges the Deployment to the desired state, increasing the number of replicas.

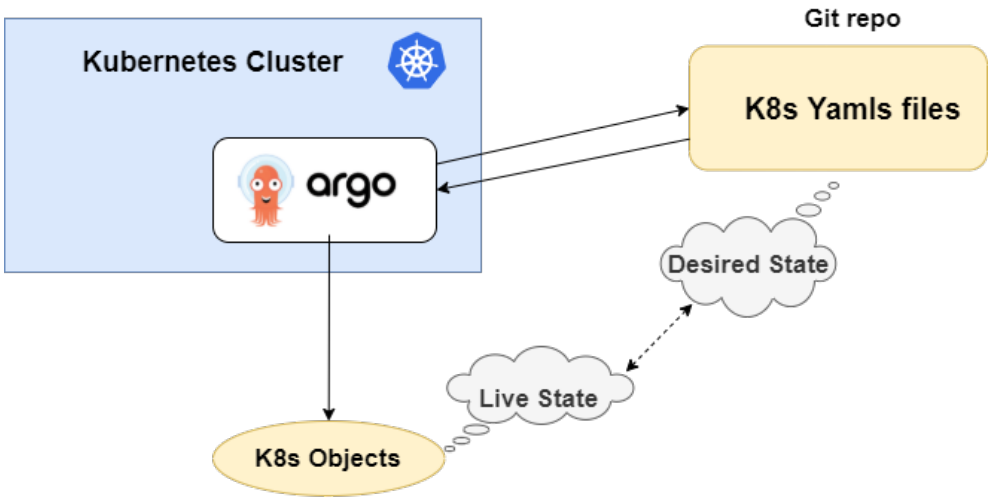


Figure B.1: ArgoCD keeps the live state synchronised with the desired state

ArgoCD is implemented to monitor the running application as a Kubernetes controller and to compare the current status against the intended target state, where yaml files are represented in the Git repository in the desired state. When an app is used differently from the planned target state, "OutOfSync" is defined. On the other side, it is characterized as 'sync' when there are no variances. The syncing procedure can be automatic or manual, for each application the user can pick his preferred mode (e.g. Application1 can be synchronized automatically and Application2 manually).

B.3 Why ArgoCD?

B.3.1 ArgoCD vs other solutions

There are several technologies now, including GitLabCI, GitHub Actions and Jenkins, which may be used for the general CI/CD. However, these technologies came before the GitOps approach. Initially, GitOps was developed to handle files with infrastructure exclusively via Git by using the existing IaC. As a consequence of a commit, the live state should converge to the intended one when the Git repository changes. The deployment procedure is thereby isolated from the CI process, as the result of a Git repository update and not necessarily a natural CI consequence. ArgoCD, Flux and Jenkins X instead, follow the GitOps paradigm.

Flux is described as a GitOps operator for Kubernetes that synchronises the state of manifests in a Git repository to what is running in a cluster. But it can only observe one repository, which makes the tool a bit awkward if we want to manage a huge number of applications.

Jenkins X collects many tools to create a development process around GitHub repositories. It can also run pipelines for continuous integration. It manages deployments

based on Git repository modifications.

ArgoCD can handle numerous application deployments in separate clusters. It controls access and permissions for teams and projects in the cluster, as well as cluster-wide rights. It features a user-friendly online interface that is both comprehensive and practical. It may be used for a variety of purposes, including monitoring, provisioning, and other tasks. ArgoCD also has several extra features that we might find handy.

ArgoCD was chosen for several reasons:

- **Web UI:** Users may monitor and interact with applications using the user interface, which is simple and expressive.
- **Multi-tenancy:** It is the one that best controls the multi-tenancy of all the systems.
- Trivial installation and configuration.
- We can handle numerous Git repositories and deploy to numerous clusters with only one instance and minimum setup.

B.3.2 Features

The ArgoCD program provides a plethora of features and settings that may be customized. The following are the most intriguing:

- External clusters can be bound to ArgoCD after it has been deployed in one k8s cluster. Then, we may choose which clusters will get the k8s objects from the Git repository specified by the ArgoCD Application.
- Web UI is a really basic and straightforward interface that allows us to accomplish several tasks, such as create or delete an application. It is also handy to monitor applications in real time.
- The history of the deployed infrastructure is preserved thanks to Git. Using commits, it is possible to revert to a previous state. It is also possible to do it directly from the UI.
- Policies for RBAC and multi-tenancy.
- Management and templating tools such as Kustomize, Helm, Jsonnet, and Ksonnet are supported.
- Prometheus metrics are exposed.
- A single instance of ArgoCD may manage many applications from various teams. A Project object can include several Applications, and this abstraction is consistent with the team logic notion. Members of a team will only view the Projects that have been allocated to them, and then just the apps that are connected with those Projects. This approach closely resembles the k8s namespaces.

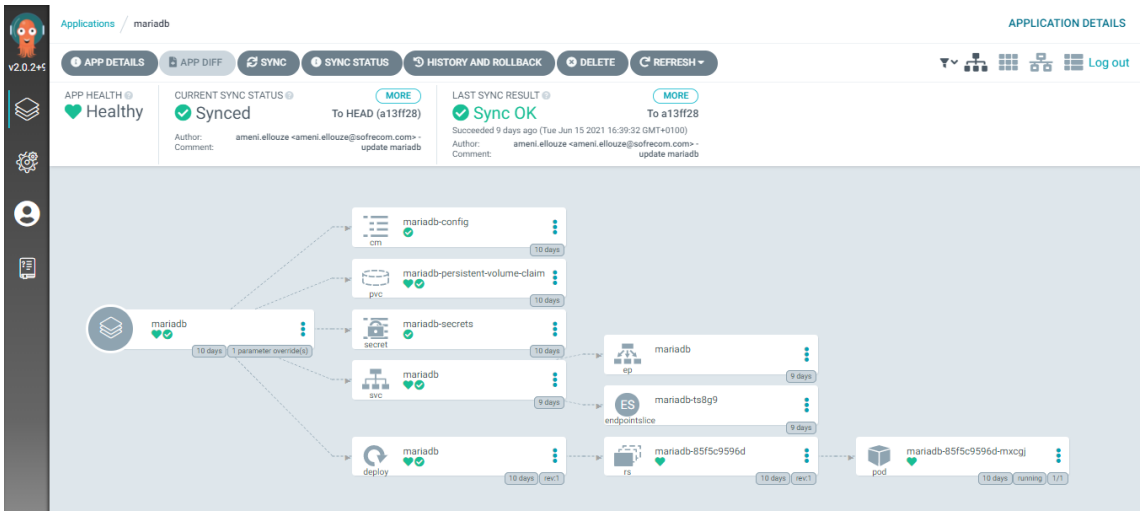


Figure B.2: ArgoCD web UI example

B.3.3 ArgoCD Architecture

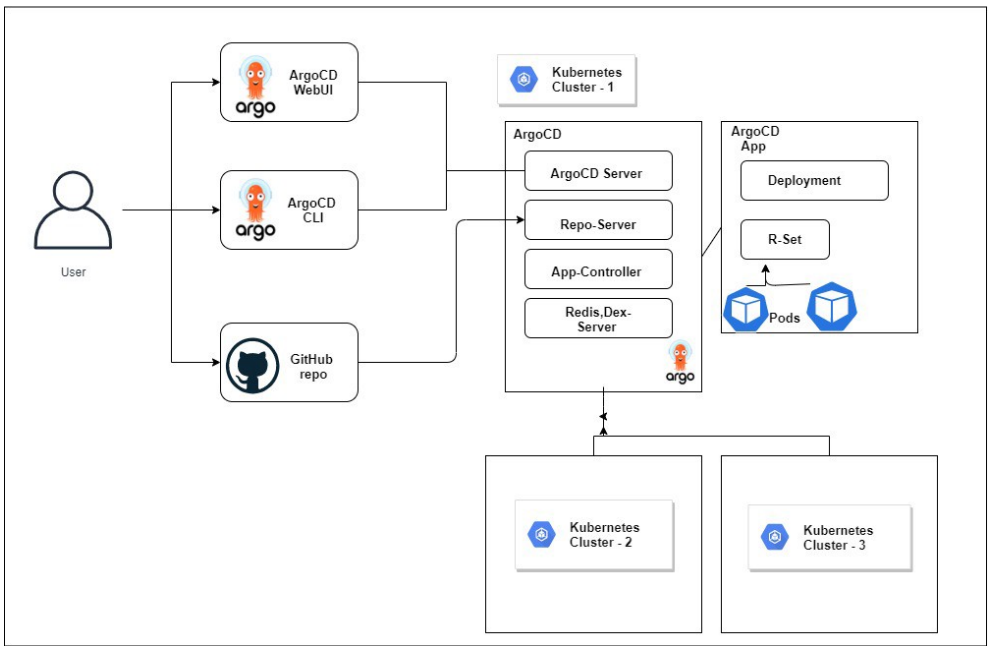


Figure B.3: ArgoCD Architecture

Once argoCD has been deployed in the Kubernetes cluster, it will have its own namespace called “argocd”, running: ”kubectl get pods -n argocd” The output is:

```
[root@master-node ~]# kubectl get pods -n argocd
NAME                                  READY   STATUS    RESTARTS   AGE
argocd-application-controller-0       1/1     Running   0          10d
argocd-dex-server-9dc558f5-stmwh      1/1     Running   0          10d
argocd-redis-759b6bc7f4-jnwjz         1/1     Running   0          10d
argocd-repo-server-5fbf484547-bth68   1/1     Running   1          10d
argocd-server-6d4678f7f6-8qb2d        1/1     Running   0          10d
```

Figure B.4: ArgoCD Pods

ArgoCD Server

The API server, which is a gRPC/REST server, is represented by the pod. It exposes the API, which may be used by the CLI, Web UI, and CI/CD systems.

ArgoCD Repo Server

It is an internal service. It maintains a local cache of the Git repository, holding manifests that describe applications.

ArgoCD Application Controller

It is a Kubernetes controller that compares the live state to the desired one given the deployed apps. In order to sync the live state, it also performs remedial measures.

ArgoCD Dex Server

It makes use of an in-memory database.

ArgoCD Redis

Redis is an in-memory data structure store that is open source (BSD licensed) that may be used as a database, cache, and message broker. It is pre-configured in this scenario with the knowledge of just three total redis servers/sentinels.

B.4 Application of Applications

This is a design that lets one app produce other applications, which may then build other apps. This enables the user to manage a collection of apps declaratively that can be deployed and configured together. It might be beneficial if several environments connected to the same application need to be deployed.
The intention is to use an Application resource manifest. This Application, marked by the letter X, points to a relative path in a remote Git repository containing one or more Application manifests. Every Application manifest included within this repository links to other relative routes contained inside other repositories.
As a result, when ArgoCD delivers the X application, it also deploys all of the other apps. When a manifest is created, removed, or modified within the repository pointed by X, ArgoCD handles adding, deleting, or updating the Application on Kubernetes in the live state.

B.5 High Availability

Argo CD is mostly stateless; all data is saved as Kubernetes objects, which are then stored in Kubernetes' etcd. Redis is only used as a temporary cache and can be deleted. If it is lost, it will be rebuilt without interruption of service. For users who want to operate Argo CD in a highly available mode, a collection of HA manifests is supplied. This starts additional containers and activates Redis in HA mode.